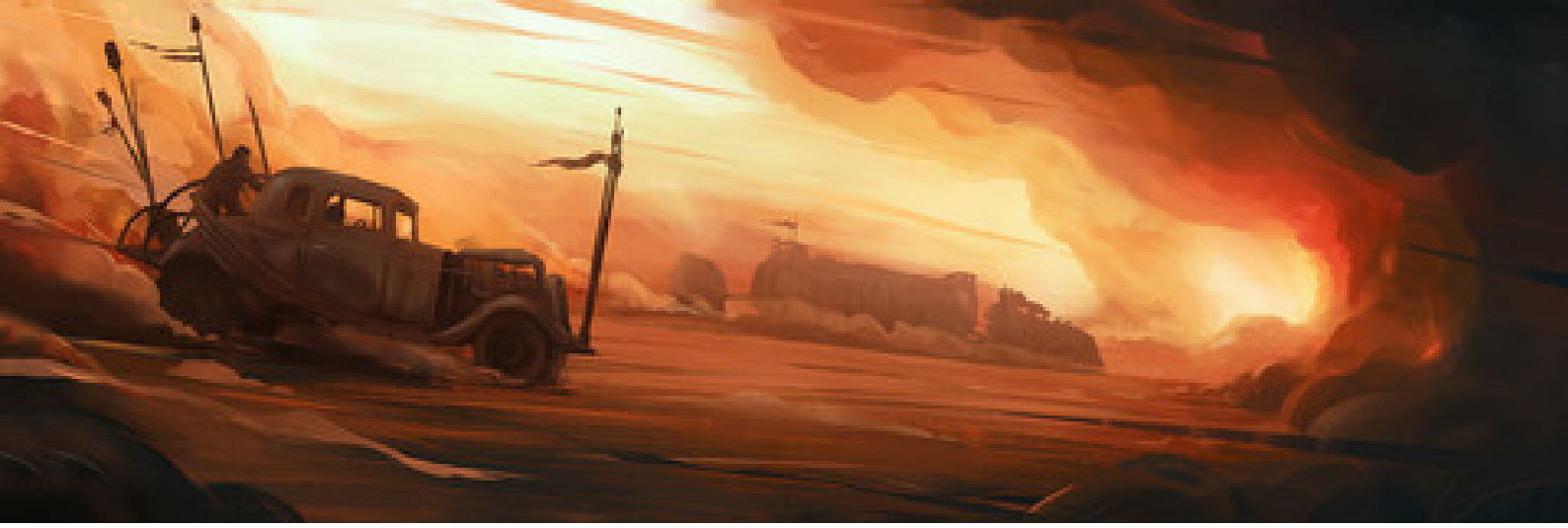
Éléments de programmation système (Télécom — ETE7-IFON1)

Part. Deux — Correction des exercices

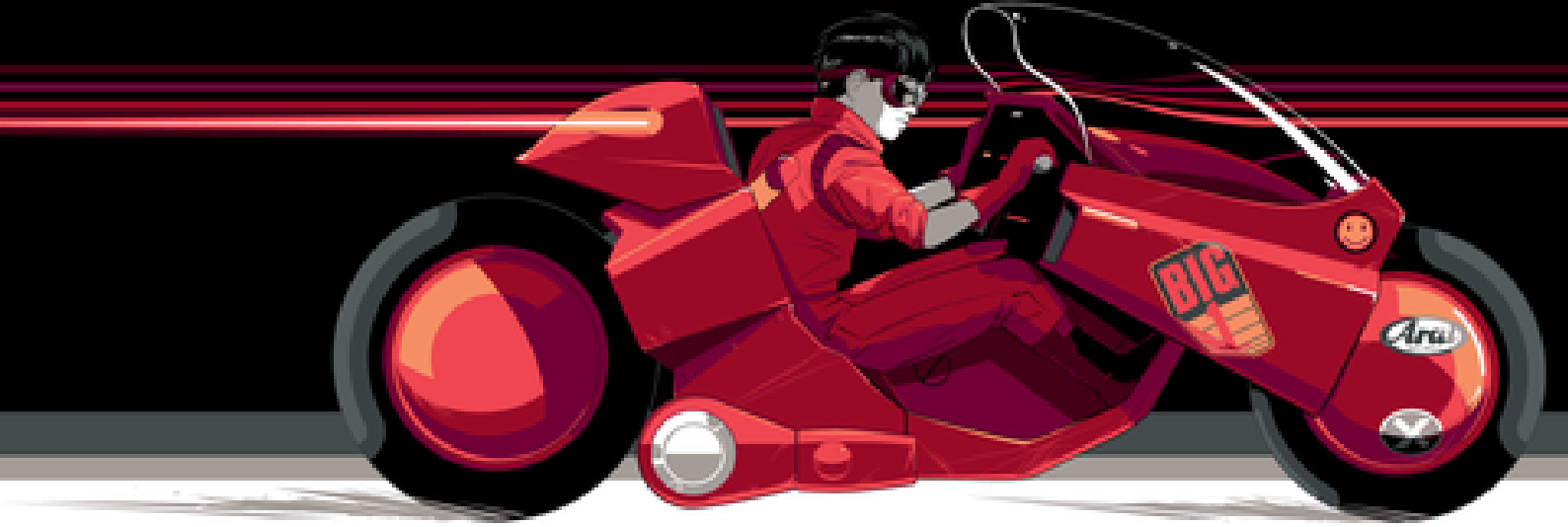
Année 2026 — 2027

Guillaume Mercier

Copyright © 2026 Guillaume Mercier
La diffusion —même partielle— de ce document, quel qu'en soit le format (physique ou non) est interdite sans l'accord de son auteur. Les illustrations sont © de leurs auteurs respectifs.



1. Premier programme



2. Utilisation des fichiers

2.1 Notion de Système de fichiers

2.2 Structures du système pour la gestion des fichiers

Exercice 1 (⋆⋆)
Si la structure d'I-Nœud contient un tableau avec les adresses des blocs de données sur le disque et que la taille d'un fichier est susceptible de changer au cours du temps, comment peut-on faire pour stocker ce tableau sans que la taille de la structure d'I-Nœud elle-même ne change ?

Correction

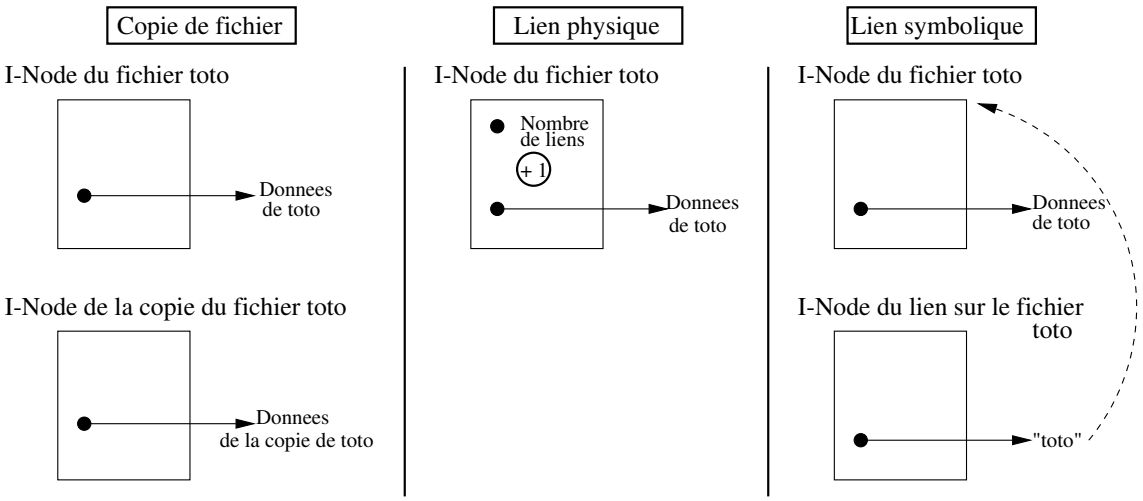
Utiliser des indirections.

2.3 Nommage et partage de données

Exercice 2 (⋆⋆)
Quelles sont les différences, d'un point de vue des structures utilisées, entre copies de fichiers, liens physiques et liens symboliques ?

Correction

— Copie de fichier : un *nouvel* I-nœud est créé avec un pointeur sur une *copie* des données du fichier ;
— Lien physique : le nombre de liens sur le fichier est incrémenté. Pas de nouvelle strcuture d'I-nœud allouée ;
— Lien symbolique : un *nouvel* I-nœud est créé pour le lien symbolique qui est un type de fichier particulier (en tant que fichier, le lien doit donc posséder son propre I-nœud). Les données du lien sont en fait le nom du fichier sur lequel il pointe (une chaîne de caractères).



2.4 Interface pratique

Exercice 3 (⋆⋆⋆)

Quelle est l'utilité du compte de référence de la structure de fichier ouvert puisqu'il semblerait qu'il ne puisse valoir qu'un ?

Correction

En fait, il peut valoir plus que 1, dans les cas de `fork` ou `dup`.

Exercice 4 (⋆)

Créez un fichier dans votre répertoire de travail (avec une commande shell). Écrivez un programme qui ouvre ce fichier et affiche la valeur (de l'index) de descripteur de fichier. Cette valeur est-elle conforme à vos attentes ?

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5
6  int main(int argc, char *argv[])
7  {
8    int fd = open("toto.txt",O_RDONLY);
9
10   fprintf(stdout,"valeur de l'index = %i\n",fd);
11
12   exit(EXIT_SUCCESS);
13 }
```

Exercice 5 ($\star$)

Écrivez un programme qui ouvre un fichier en lecture seule et qui crée le fichier en mode lecture/écriture/exécution (uniquement pour vous). On supposera que ce fichier n'existe pas déjà.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5
6  int main(int argc, char *argv[])
7  {
8   int fd = open("toto.txt", O_RDONLY | O_CREAT, S_IRWXU );
9
10  fprintf(stdout,"valeur de l'index = %i\n",fd);
11
12  exit(EXIT_SUCCESS);
13 }
```

Exercice 6 (⋆)

Écrivez un programme qui écrit **un entier** dans un fichier (i.e. pas sous forme de chaîne de caractères). Dans un terminal, affichez le contenu de ce fichier (à l'aide d'une commande shell comme `cat`, `less` ou `more` et non pas d'un programme C).

— Que constatez-vous ?

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  int fd     = -1;
  int valeur = 33;
  int ret    = -1;

  fd = open("toto.txt", O_RDWR | O_CREAT, S_IRWXU);
  assert( fd != -1);

  ret = write(fd, &valeur, sizeof(valeur));
  assert( ret != -1);

  close(fd);
  exit(EXIT_SUCCESS);
}
```

Exercice 7 (⋆)

Modifiez le programme précédent et rajoutez-y la lecture (et l'affichage) de l'entier que vous venez d'écrire dans le fichier.

Correction

On doit réutiliser le descripteur pour lire l'entier parce que la position courante après l'écriture est en fin de fichier. En fait, il serait plus efficace de modifier la position courante avec un appel à `lseek` mais cette fonction n'est pas connue au moment où cet exercice est fait.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  int fd      = -1;
```

```
11    int valeur  = 33;
12    int valeur2 = -1;
13    int ret     = -1;
14
15    fd = open("toto.txt", O_RDWR | O_CREAT, S_IRWXU);
16    assert(fd != -1);
17
18    ret = write(fd, &valeur, sizeof(valeur));
19    assert(ret != -1);
20
21    close(fd);
22
23    fd = open("toto.txt", O_RDONLY);
24    assert(fd != -1);
25
26    ret = read(fd, &valeur2, sizeof(valeur2));
27    assert(ret != -1);
28
29    fprintf(stdout,"Valeur lue = %i\n",valeur2);
30
31    close(fd);
32
33    exit(EXIT_SUCCESS);
34 }
```

Exercice 8 ($\star$)

Écrivez un programme qui écrit n caractères dans un fichier. Les arguments du programme seront le nom du fichier et la valeur de n. Le i-ème caractère devra être écrit à la position 2^i dans le fichier, en supposant que la numérotation des caractères commence à 1.

Correction

Note : une solution alternative possible serait d'utiliser la fonction `pwrite` qui permet d'écrire directement à la position courante prise en dernier argument.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/stat.h>
6  #include <fcntl.h>
7  #include <assert.h>
8  #include <errno.h>
9
10 int main(int argc, char *argv[])
11 {
12   int   fd  = -1;
13   off_t pos = 2;
14   char  val = 0;
15
16   assert(3 == argc);
17
18   fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, S_IRWXU);
19
20   for(int i = 1 ; i <= atoi(argv[2]) ; i++ , pos *= 2){
21     val = 'a';
22     lseek(fd, pos, SEEK_SET);
23     fprintf(stdout,"Write %c at pos %li\n",val,pos);
24     write(fd, &val, sizeof(char));
25   }
26
27   close(fd);
28
29   return 0;
30 }
```

Exercice 9 ($\star$)

Écrivez un programme qui redirige tous ses appels à `printf` dans un fichier `output.txt` plutôt que sur sa sortie standard, sans modifier les appels à `printf` dans le programme d'origine.

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <assert.h>

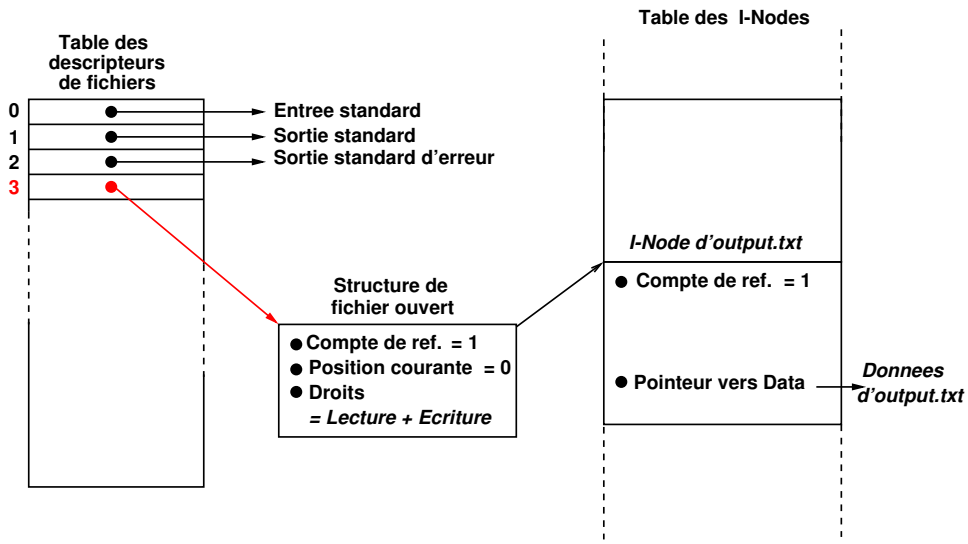
int main(int argc, char *argv[])
{
  int fd = -1;

  fd = open("output.txt", O_RDWR | O_CREAT , S_IRWXU);
  assert(fd != -1);

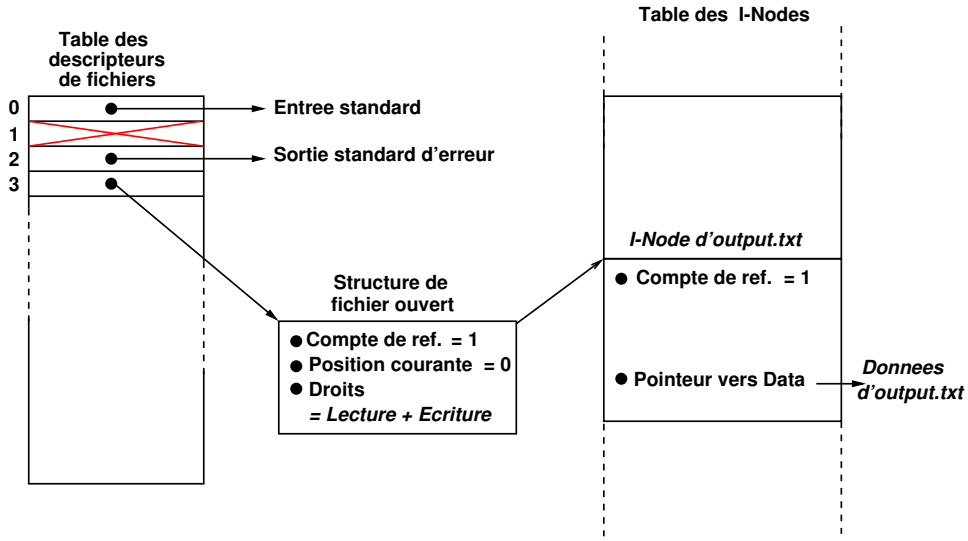
  close(STDOUT_FILENO);
  dup(fd);
  close(fd);

  printf("Hello World!\n");

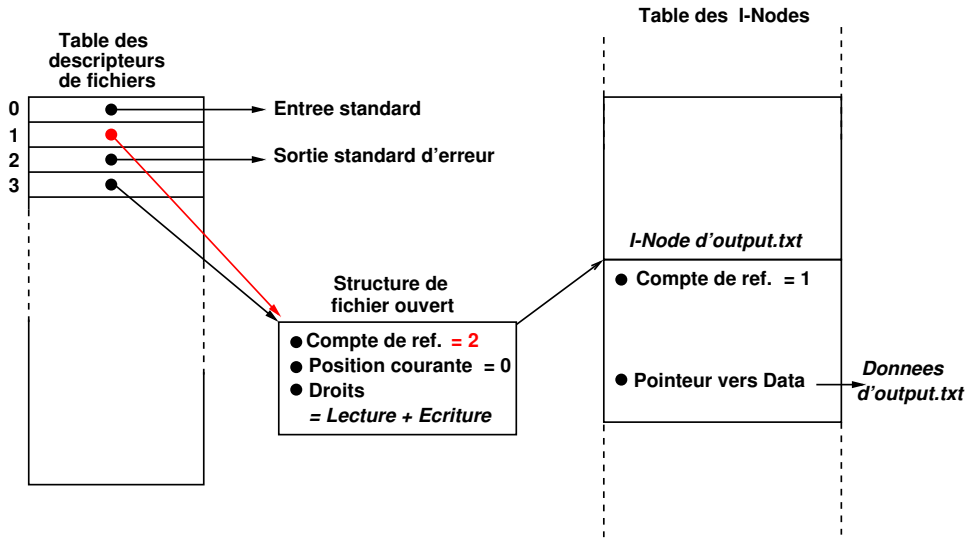
  exit(EXIT_SUCCESS);
}
```



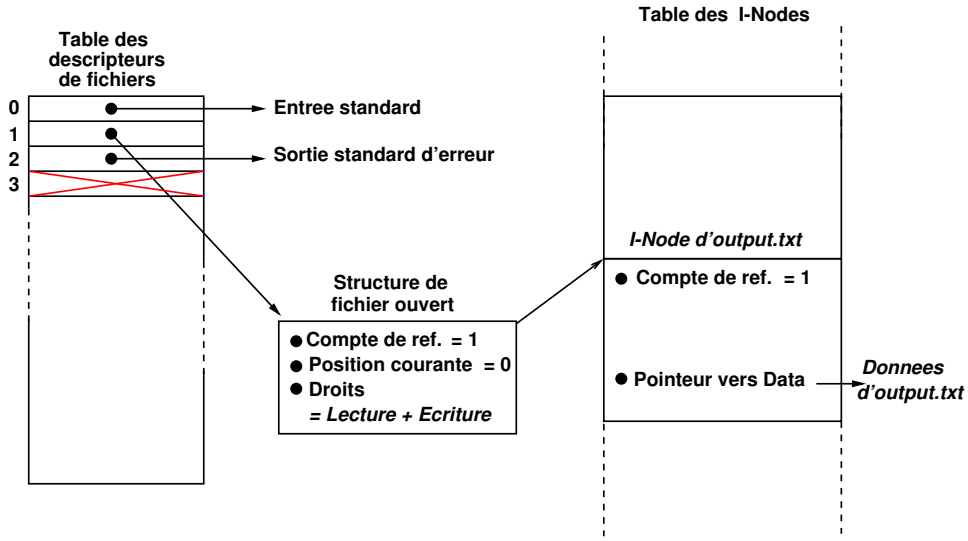
Exécution de la ligne 12



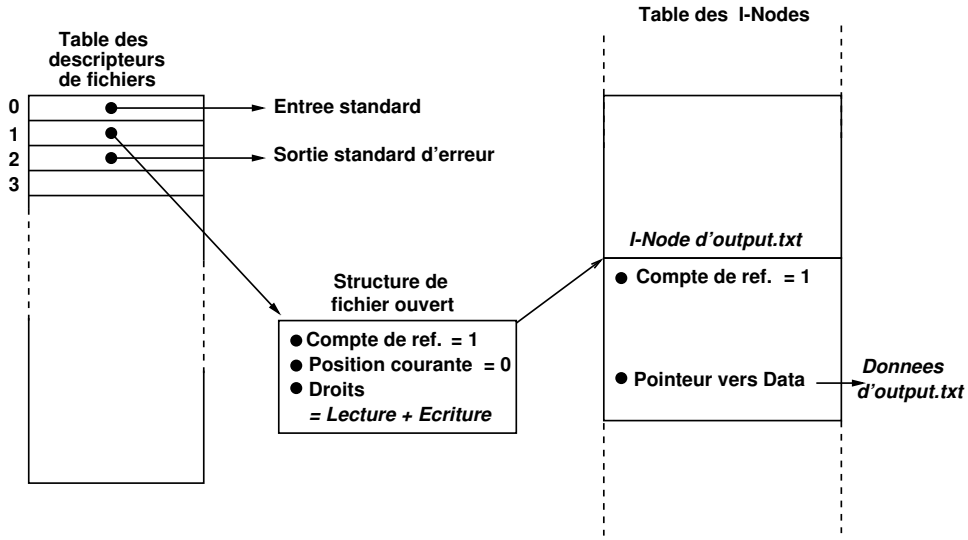
Exécution de la ligne 15



Exécution de la ligne 16



Exécution de la ligne 17



Situation finale

Exercice 10 ($\star$)

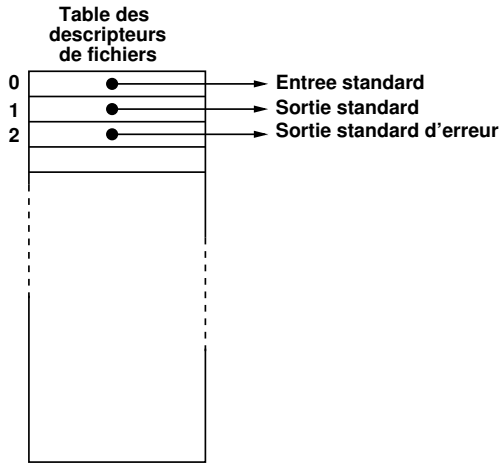
Que se passera-t-il si un programme effectue les instructions suivantes ?

```
1 close(STDOUT_FILENO);
2 open("toto.txt",O_RDWR);
```

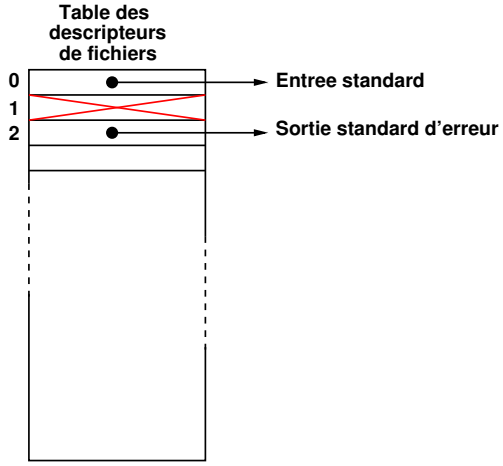
NB : On supposera que le fichier `toto.txt` *existe.*

Correction

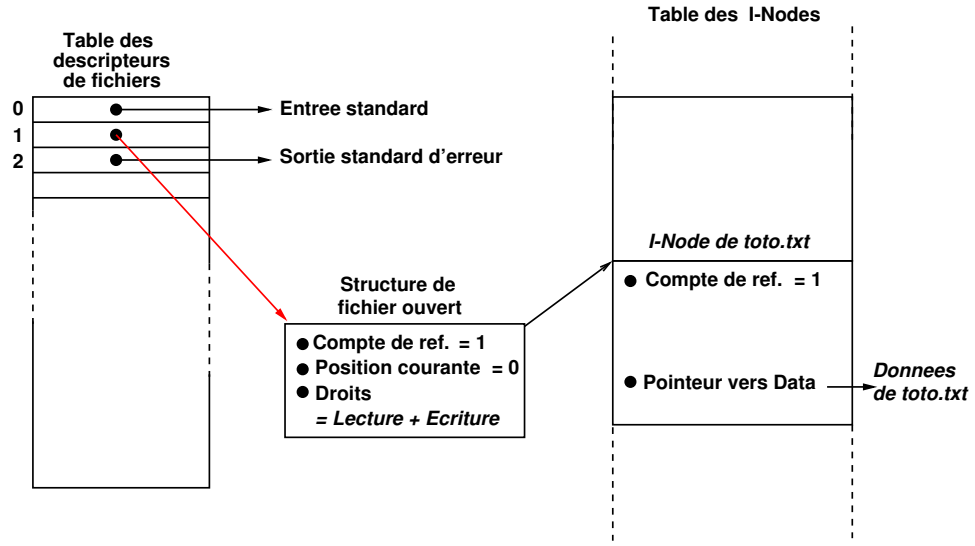
Ces deux lignes de code établissent une redirection de la sortie standard vers le fichier toto.txt. Fermer `STDOUT_FILENO` créé un « trou » dans la table des descripteurs de fichiers du processus appelant. Ledit « trou » est ensuite réutilisé lorsque un nouveau descripteur doit être alloué au moment de l'ouverture du fichier toto.txt.



Situation initiale



Exécution de la ligne 1



Exécution de la ligne 2

Exercice 11 ($\star$)

Écrivez un programme qui ouvre un fichier avec la *fonction* `fopen` puis qui ouvre un fichier (le même ou un autre, peu importe) avec l'*appel-système* `open`. Affichez la valeur de l'index de descripteur associé.

— Que constatez-vous ?

— Que pouvez-vous en déduire ?

Correction

L'index du descripteur est 4, ce qui signifie qu'un autre descripteur a été alloué, pendant le `fopen` a priori.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <errno.h>
6  #include <assert.h>
7
8  int main(int argc, char *argv[])
9  {
10   FILE *ptr = NULL;
11   int   fd  = -1;
12
13   ptr = fopen("output.txt","r");
14   assert(ptr);
15
16   fd = open("output.txt", O_RDWR);
17   assert(fd != -1);
```

```
18
19    fprintf(stdout,"Valeur de l'index = %i\n", fd);
20
21    close(fd);
22    fclose(ptr);
23
24    exit(EXIT_SUCCESS);
25 }
```

Exercice 12 ($\star$)

Écrivez un programme qui redirige tous ses appels à `fprintf(stdout, ...)` dans un fichier `output.txt` plutôt que sur sa sortie standard, sans modifier les appels à `fprintf` dans le programme d'origine et sans utiliser des (index de) descripteurs de fichiers ni la duplication.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char *argv[])
5  {
6    FILE *fd = NULL;
7
8    fprintf(stdout,"Coucou - 1\n");
9
10   fd = freopen("output.txt","w+",stdout);
11
12   fprintf(stdout,"Coucou - 2 \n");
13
14   fclose(fd);
15
16   fprintf(stdout,"Coucou - 3 \n");
17
18   exit(EXIT_SUCCESS);
19 }
```

2.5 Exercices

Exercice 13 ($\star\star$)

Écrivez un programme qui lit l'*intégralité* d'un fichier, caractère par caractère (i.e. octet par octet) et qui affiche le tout sur sa sortie standard d'*erreur*.
N.B L'utilisation de `stat` ou `fstat` n'est pas autorisée pour cet exercice.

Correction

Solutions possibles :

— Version 1 : tester le code de retour de `read` : 0 signifie que la fin de fichier est atteinte
— Version 2 : tester le caractère lu : `EOF` est le caractère de fin de fichier. Attention : ce caractère n'est pas toujours systématiquement disponible, il faut vérifier s'il est disponible pour la fonction utilisée. Par exemple, il existe pour `getc`, mais pas pour `read`...
— Version 3 : tester la fin de fichier avec la fonction `feof`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <assert.h>

#if VERSION == 1
int main(int argc, char *argv[])
{
  int fd = -1;
  char tmp;

  fd = open("input.txt",O_RDONLY);

  while(read(fd, &tmp, sizeof(char))){
    fprintf(stderr,"%c",tmp);
  }

  close(fd);
  exit(EXIT_SUCCESS);
}

#elif VERSION == 2
int main(int argc, char *argv[])
{
  FILE *file = NULL;
  char tmp;

  file = fopen("input.txt","r");

  // tmp = getc(file) est equivalent a
  // fread(&tmp,sizeof(char),1,file);
  // getc permet de tester un caractere
  // particulier, le caractere EOF (End-Of-File)

  while( (tmp = getc(file)) != EOF){
    fprintf(stderr,"%c",tmp);
  }

  fclose(file);
  exit(EXIT_SUCCESS);
}

#else // VERSION == 3
int main(int argc, char *argv[])
{
  FILE *file = NULL;
  char tmp;
```

```
50
51    file = fopen("input.txt","r");
52
53    tmp = getc(file);
54    while( !feof(file)){
55      fprintf(stderr,"%c",tmp);
56      tmp = getc(file);
57    }
58
59    fclose(file);
60    exit(EXIT_SUCCESS);
61  }
62
63  #
64  #endif
```

Exercice 14 (⋆⋆)

On suppose l'existence d'un fichier texte contenant *un* mot par ligne avec plusieurs lignes, par exemple :

```
toto
titi
tata
```

Écrivez un programme qui :

1. ouvre ce fichier en lecture seule ;
2. compte le nombre de lignes dans le fichier ;
3. alloue un tableau dynamiquement pour contenir les différents mots (on suppose que leur taille fait au maximum `MAX_SIZE`). La taille de ce tableau doit correspondre *exactement* au nombre de mots (i.e. de lignes) dans le fichier ;
4. stocke les mots dans le tableau ;
5. affiche le contenu de ce tableau.

Correction

Pas de correction pour cet exercice qui sera réutilisé en projet Système & Réseaux.

2.6 Travaux Pratiques

Pas de correction pour cette partie.

Exercice 15 (⋆)

Écrivez une version en **C** de la commande `cat` en utilisant uniquement les fonctions standards d'entrées/sorties de la bibliothèque du **C** `fread` et `fwrite`, et en lisant et écrivant des blocs d'**un** caractère (oui, oui, un seul caractère ...).

Correction

Exercice 16 ($\star$)
Écrivez une seconde version de `cat` en utilisant uniquement les appels-système `read` et `write`.

Correction

Exercice 17 ($\star$)
En utilisant la commande interne `time` du shell `bash`, comparez les performances des deux versions pour une lecture/écriture d'un fichier raisonnablement volumineux (i.e. de plus de 1 Mo).
Exemple :

```
$ time ./my_cat /lib/libc.a > tst
  real    0m8.658s
  user    0m2.920s
  sys     0m5.740s
$
```

Interprétez les résultats obtenus. Quelles sont les commandes de plus bas niveau : `read/write` ou `fread/fwrite` ?

Correction

Exercice 18 ($\star$)
Essayez le programme suivant :

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
  fwrite("Hello", sizeof(char), 6, stdout);
  write(STDOUT_FILENO, "World !", 7);
  exit(EXIT_SUCCESS);
}
```

L'observation est-elle en accord avec votre interprétation des résultats précédents ?

Correction



3. Du programme au processus

3.1 Exécution d'un programme

3.2 Identification d'un processus

Exercice 19 ($\star$)
Écrivez un programme qui affiche le PID de son processus.

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  pid_t pid = -1;

  pid = getpid();

  fprintf(stdout,"Mon pid est = %i\n",pid);

  exit(EXIT_SUCCESS);
}
```

3.3 Création de processus

Exercice 20 ($\star$)

Écrivez un programme qui crée un processus-enfant et affiche ensuite une chaîne de caractères.

— Que constatez-vous ?

— Est-ce cohérent par rapport à ce que vous connaissez du fonctionnement de `fork` ?

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <errno.h>
6  #include <assert.h>
7
8  int main(int argc, char *argv[])
9  {
10   fork();
11
12   fprintf(stdout,"Hello World!\n");
13
14   exit(EXIT_SUCCESS);
15 }
```

La chaîne `"Hello World!"` est affichée deux fois car tout le code situé après l'appel à `fork` est exécuté simultanément par le processus-parent et son enfant.

Exercice 21 ($\star\star$)

Reprenez le programme de l'exercice précédent et rajoutez un affichage de chaîne de caractères *avant* l'appel à `fork`.

— L'affichage est-il en accord avec vos attentes ? Pourquoi ?

— Que se passe-t-il lorsque il n'y a pas de caractère `\n` à la fin de la chaîne affichée par le `printf` placé avant le `fork` ?

Correction

Soit le code suivant :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <errno.h>
6  #include <assert.h>
7
8  int main(int argc, char *argv[])
9  {
10   fprintf(stdout,"Hello World\n");
11
12   fork();
13
```

```
14    fprintf(stdout,"Hello World\n");
15
16    exit(EXIT_SUCCESS);
17  }
```

L'affichage est alors :

```
Hello World
Hello World
Hello World
```

Le premier `"Hello World"` est affiché par le parent et les deux suivants sont affichés par le parent et l'enfant (impossible de savoir dans quel ordre).

Si on enlève le caractère `\n` à la fin de la première chaîne, on aura alors comme affichage :

```
Hello WorldHello World
Hello WorldHello World
```

En effet, la première chaîne `"Hello World"` est écrite dans un buffer dans la mémoire du processus. Le contenu de ce buffer n'est pas affiché à l'écran car il n'y a pas de `\n` à la fin de la chaîne (ni d'appel à `fflush` après le `printf`). Au moment du `fork`, la mémoire du processus-parent est dupliquée dans le processus-enfant qui possède donc la chaîne `"Hello World"` dans son propre buffer. Au moment du second appel à `printf`, on écrit une nouvelle fois la chaîne `"Hello World"` dans le buffer puis on l'affiche (à cause du `\n` en fin de chaîne). Ceci se produit à la fois dans le parent et dans l'enfant, d'où l'affichage.

Exercice 22 (⋆⋆)
Écrivez un programme qui crée **exactement** 10 processus-enfants.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <errno.h>
6  #include <assert.h>
7
8  int main(int argc, char *argv[])
9  {
10   for(int index = 0 ; index < 10 ; index++){
11     pid_t child = fork();
12     if(0 == child){
13       break;
14     } else {
15       fprintf(stdout,"creation enfant %i\n",child);
16     }
17   }
18   exit(EXIT_SUCCESS);
19 }
```

Il est impératif que les processus-enfants sortent de la boucle (instruction `break`) sinon un nombre bien plus important de processus sera créé et non pas 10 (exactement).

3.4 Identifier le parent d'un processus

Exercice 23 (⋆)
Écrivez un programme qui crée un processus-enfant puis affiche le PID de ce dernier tandis que le processus-enfant affichera le PID de son parent. La chaîne à afficher sera donc différente selon que l'on se trouve dans le parent ou dans l'enfant.

Correction

On utilise le code de retour de `fork` pour savoir si on se trouve dans le processus-parent ou dans le processus-enfant :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <errno.h>
6  #include <assert.h>
7
8  int main(int argc, char *argv[])
9  {
10   pid_t child = fork();
11
12   if(0 == child){
13     fprintf(stdout,"Enfant: pid parent = %i\n",getppid());
14   } else if (child > 0){
15     fprintf(stdout,"Parent: creation de l'enfant de pid
         %i\n",child);
16   }
17
18   exit(EXIT_SUCCESS);
19 }
```

Exercice 24 (⋆)
Écrivez un programme qui affiche son PID puis crée un processus-enfant. Le processus-parent affiche ensuite le PID de son enfant tandis que son enfant affichera son propre PID puis celui de son parent. Vérifiez la cohérence de l'affichage.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <errno.h>
6  #include <assert.h>
7
8  int main(int argc, char *argv[])
```

```
9  {
10   pid_t child = -1;
11
12   fprintf(stdout,"Parent: je suis : %i\n", getpid());
13
14   child = fork();
15
16   if( child > 0){
17     fprintf(stdout,"Parent: Child : %i\n", child);
18   } else if (0 == child){
19     fprintf(stdout,"Enfant: je suis : %i\n", getpid());
20     fprintf(stdout,"Enfant: Parent : %i\n", getppid());
21   }
22
23   exit(EXIT_SUCCESS);
24 }
```

```
L'affichage est conforme à nos attentes~:
mercierg@Palamede$ ./Exercice_fork_affichage_pid_ppid
Parent: je suis : 520026
Parent: Child : 520027
Enfant: je suis : 520027
Enfant: Parent : 520026
```

Cependant, l'affichage peut apparaître incohérent, avec l'enfant qui affiche un pid pour son parent ne correspondant pas à ce qu'il devrait être, par exemple :

```
mercierg@Palamede$ ./Exercice_fork_affichage_pid_ppid
Parent: je suis : 520028
Parent: Child : 520029
Enfant: je suis : 520029
Enfant: Parent : 2510
```

L'explication se trouve en Section 3.7 (du volume 1).

3.5 Héritage inter-processus

Exercice 25 (⋆⋆)
Soit un processus qui ouvre un fichier régulier quelconque. Ce processus crée ensuite un processus-enfant à l'aide de `fork()`. Faites le schéma résumant l'état des structures de données : tables des descripteurs de fichiers, structures de fichiers ouverts et Table des I-Nœuds suite à ces opérations.

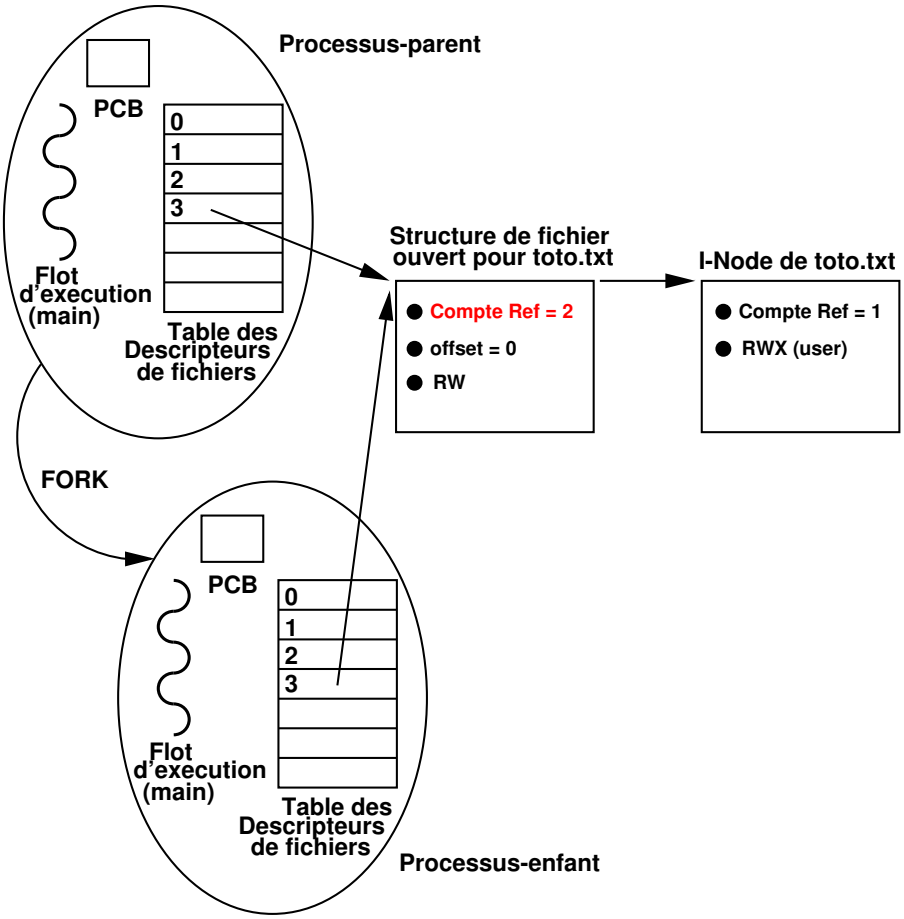
Correction

Soit la succession d'instructions suivante :

```
1  int main(int argc, char *argv[]){
2
3    int fd = open("toto.txt", O_RDWR | O_CREAT, S_IRWXU);
4
5    pid_t child = fork();
```

```
6
7     /* reste du code */
8
9     exit(EXIT_SUCCESS);
10 }
```

Voilà ce qui se passe au niveau des structures de données :



C'est le second cas où le compte de référence d'une structure de fichier ouvert peut être supérieur à 1 (le premier cas étant la duplication de descripteur).

3.6 Modèle *fork/join*

Exercice 26 (⋆)

Écrivez un programme qui :

1. crée un processus-enfant
2. effectue une petite boucle d'attente active, par exemple :
   ```
   do{int toto = 0;
   while(++toto);
   }while(0);
   ```
3. se synchronise avec cet enfant pour sa terminaison

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/wait.h>
6  #include <errno.h>
7  #include <assert.h>
8
9  int main(int argc, char *argv[])
10 {
11   pid_t child = fork();
12
13   if( child > 0){ // code du parent
14     short int toto = 0;
15     pid_t fini = -1;
16
17     fprintf(stdout,"Parent : je suis : %i | pid enfant %i\n",
           getpid(), child);
18
19     while(++toto); // attente active
20
21     fini = wait(NULL);
22
23     fprintf(stdout,"Parent : l'enfant %i est fini\n", fini);
24   }
25
26   exit(EXIT_SUCCESS);
27 }
```

Exercice 27 ($\star$)

Reprenez le programme précédent et affichez dans le parent le PID de l'enfant terminé, ainsi que son code de retour. Le processus-enfant affichera son PID, celui de son parent puis attendra un petit peu avant d'appeler `exit`. Vous donnerez une valeur de retour bien identifiable pour l'enfant (i.e ni 0, 1 ou -1).

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/wait.h>
6  #include <errno.h>
7  #include <assert.h>
8
9  int main(int argc, char *argv[])
10 {
11   pid_t child = fork();
12
13   if(0 == child){ // code de l'enfant
14     short int toto = 0;
15
16     fprintf(stdout,"Enfant : Je suis  %i\n",getpid());
17     fprintf(stdout,"Enfant : pid parent %i\n",getppid());
18
19     while(++toto);
20
21     exit(33);
22   } else if (child > 0){ // code du parent
23     pid_t fini = -1;
24     int status = 0;
25
26     fprintf(stdout,"Parent : creation enfant %i\n",child);
27
28     fini = wait(&status);
29     fprintf(stdout,
30             "Parent : le processus %i est fini, valeur de
                retour = %i\n",
31             fini,
32             WEXITSTATUS(status));
33   }
34   exit(EXIT_SUCCESS);
35 }
```

Exercice 28 (⋆⋆)

Écrivez un programme qui crée quelques processus-enfants, puis fait une boucle d'attente active et enfin les attend dans l'ordre décroissant de leur valeur de PID. Les processus-enfants feront quant à eux une boucle d'attente (active ou passive) de durée variable selon leur PID.

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_ENFANT 5

int main(int argc, char **argv)
{
  pid_t child[NUM_ENFANT];
  int toto = 0;

  for(int idx = 0 ; idx < NUM_ENFANT ; idx++){
    child[idx] = fork();

    if( 0 == child[idx]){
      pid_t pid = getpid();
      usleep(pid+1000);
      exit(33+idx);
    }
    else{
      fprintf(stdout,"===== Creation de l'enfant :
         %i\n",child[idx]);
    }
  }

  while(++toto);

  for(int idx = NUM_ENFANT - 1 ; idx >= 0 ; idx--){
    pid_t child2 = waitpid(child[idx],NULL,0);
    fprintf(stdout,"Enfant %i termine \n",child2);
  }

  return EXIT_SUCCESS;
}
```

3.7 Terminaison d'un processus

Exercice 29 (⋆⋆⋆)
Pouvez-vous donner un type de programme qui ne fonctionnera pas correctement (voire pas du tout) à cause d'un nombre de zombies trop important ?

Correction

Un processus-serveur qui crée des processus pour traiter les requêtes des clients et qui ne récupère pas la terminaison de ses enfants.

Exercice 30 (⋆⋆)
Écrivez un programme qui crée un processus-enfant et qui se termine immédiatement.
Le processus-enfant devra s'endormir pendant quelques secondes puis afficher le PID de son parent.

— Exécutez ce programme plusieurs fois. Qu'observez-vous ?
— Est-ce cohérent ?

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  pid_t child = -1;

  fprintf(stdout,"Parent: je suis : %i\n", getpid());

  child = fork();

  if( child > 0){
    fprintf(stdout,"Parent: Child : %i\n", child);
  } else if (0 == child){
    fprintf(stdout,"Enfant: je suis : %i\n", getpid());
    sleep(3);
    fprintf(stdout,"Enfant: Parent : %i\n", getppid());
  }

  exit(EXIT_SUCCESS);
}
```

Le pid affiché ne correspond pas forcément à celui du processus-parent. Adoption des orphelins-zombies par init (1) ou par systemd.

Pour aller plus loin : Retour sur la synchronisation parent-enfant

3.8 États d'un processus

Exercice 31 (⋆)
Reprenez un des exercices précédents avec de l'attente active et remplacez-la par un appel à `sleep`. Qu'observez-vous ?

Correction

Ça ne change rien a priori, sauf au niveau des performances car le processus passe de l'état actif à l'état endormi/bloqué et n'occupe pas le processeur.

3.9 Recouvrement

Exercice 32 (⋆)
Créez un premier programme basique (qui effectue juste un appel à `printf` par exemple) puis créez un second programme qui effectue un recouvrement avec le code de ce premier programme.

Correction

Soit le code suivant (`hello_world.c`, compilé en exécutable `hello_world`) :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[])
5 {
6   printf("Hello World!\n");
7
8   exit(EXIT_SUCCESS);
9 }
```

Le recouvrement se fait de la façon suivante :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/wait.h>
6  #include <errno.h>
7  #include <assert.h>
8
9  int main(int argc, char *argv[])
10 {
11   char *my_argv[] = {"hello_world", NULL};
12
13   execv("hello_world", my_argv);
14
15   exit(EXIT_SUCCESS);
16 }
```

Exercice 33 (⋆⋆)
Reprenez le programme de l'exercice précédent, mais le recouvrement devra être effectué par un processus-enfant créé dans le second programme.

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <errno.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  pid_t child = fork();

  if(0 == child){ // Code de l'enfant

    char *my_argv[] = {"hello_world", NULL};

    execv("hello_world", my_argv); // recouvrement

  } else if ( child > 0) { // Code du parent

    wait(NULL); //synchronisation

  }

  exit(EXIT_SUCCESS);
}
```

Exercice 34 (⋆⋆)
Reprenez l'exercice précédent, mais le programme utilisé pour le recouvrement effectuera la somme de ses paramètres (dont on on supposera que ses arguments sont uniquement des entiers).

Correction

Soit le code du programme `somme.c`, compilé en un exécutable appelé `somme` :

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
  int somme = 0;

  while(*++argv)
    somme += atoi(*argv);
```

```
10
11    fprintf(stdout,"Somme = %i\n", somme);
12
13    exit(EXIT_SUCCESS);
14  }
```

Alors le recouvrement se fera de la façon suivante :

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <unistd.h>
4   #include <fcntl.h>
5   #include <sys/wait.h>
6   #include <errno.h>
7   #include <assert.h>
8
9   int main(int argc, char *argv[])
10  {
11    pid_t child = fork();
12
13    if(0 == child){ //Code de l'enfant
14
15      //Tableau des arguments
16      char *my_argv[] = {"somme", "1", "2", "3", NULL};
17
18      execv("somme", my_argv); // recouvrement
19
20    } else if (child > 0) { // Code du parent
21
22      wait(NULL); // synchronisation
23
24    }
25
26    exit(EXIT_SUCCESS);
27  }
```

L'appel produit bien le résultat attendu :

```
mercierg@Palamede:$ ./Exercice_recouvrement_somme
Somme = 6
```

Exercice 35 (⋆⋆)

Écrivez un programme qui ouvre un fichier quelconque et affiche la valeur (de l'index) du descripteur associé. Exécutez ce programme :

1. en le lançant directement depuis un terminal
2. en créant un processus-enfant qui ouvre d'abord un fichier (le même ou un autre, peu importe) puis qui effectue un recouvrement (via `exec`) avec le premier programme.

— Que constatez-vous ?

— L'affichage est-il cohérent ?

Correction

Pour cet exercice on peut exécuter le code de l'exercice 4 (ou 5) (ouverture d'un fichier et affichage de l'index de descripteur associé).

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/wait.h>
6  #include <errno.h>
7  #include <assert.h>
8
9  int main(int argc, char *argv[])
10 {
11   pid_t child = fork();
12
13   if(0 == child){ // Code de l'enfant
14
15     //Ouverture du fichier toto.txt, dont on suppose
         l'existence
16     int fd = open("toto.txt", O_RDONLY);
17     fprintf(stdout,"Valeur de l'index de descripteur =
         %i\n",fd);
18     fflush(stdout);
19
20     //Tableau d'arguments
21     //On reexecute le code de l'exercice 4 (ouverture de
         fichier
22     // et affichage de l'index de descripteur
23     char *my_argv[] = {"Exercice_ouverture_fichier", NULL};
24
25     execv("Exercice_ouverture_fichier", my_argv);
         //recouvrement
26
27   } else if ( child > 0) { // Code du parent
28
29     wait(NULL); // synchronisation
30
31   }
32
33   exit(EXIT_SUCCESS);
34 }
```

L'affichage est le suivant :

```
mercierg@Palamede:$ ./Exercice_fork_fichier
Valeur de l'index de descripteur = 3
valeur de l'index = 4
```

Comme un fichier a été ouvert par le processus, l'index est 3. Ensuite, on effectue un recouvrement du processus par un autre exécutable qui ouvre également un fichier. Mais comme le descripteur du

fichier précédent est conservé à travers le recouvrement, le nouvel index est 4.

Exercice 36 (⋆)
Reprenez l'exercice précédent en positionnant l'option `O_CLOEXEC` au moment de l'ouverture du fichier avant d'effectuer le recouvrement (ie. l'appel à `exec`). Que constatez-vous ?

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <errno.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  pid_t child = fork();

  if(0 == child){ // Code de l'enfant

    //Ouverture du fichier en mode CLOEXEC
    int fd = open("toto.txt", O_RDONLY | O_CLOEXEC);
    fprintf(stdout,"Valeur de l'index de descripteur =
       %i\n",fd);
    fflush(stdout);

    //Tableau d'arguments
    char *my_argv[] = {"Exercice_ouverture_fichier", NULL};
    execv("Exercice_ouverture_fichier", my_argv);
       //recouvrement

  } else if (child > 0) { // Code du parent

    wait(NULL); //synchronisation

  }

  exit(EXIT_SUCCESS);
}
```

L'affichage est le suivant :

```
mercierg@Palamede:$ ./Exercice_fork_fichier
Valeur de l'index de descripteur = 3
valeur de l'index = 3
```

Comme un fichier a été ouvert par le processus, l'index est 3. Ensuite, on effectue un recouvrement du processus par un autre exécutable qui ouvre également un fichier mais avec l'option `O_CLOEXEC`. Ce descripteur de fichier est donc fermé au moment du recouvrement et le nouvel index est donc

encore 3.

3.10 Pour aller plus loin : exécution de commandes shell dans un programme



4. Communication inter-flots

4.1 Tubes

4.1.1 Pour aller plus loin : exécution/interaction de/avec une commande shell

Exercice 37 ($\star$)
Écrivez un programme avec deux processus qui s'échangent une chaîne de caractères via un tube de communication *anonyme*.

Correction 1

Si on part du principe que le processus-parent envoie une chaîne à son enfant, alors il est écrivain et son enfant est lecteur. Cela donne le sens de circulation des données dans le tube.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9
10 int main(int argc, char *argv[])
11 {
12   pid_t child = -1;
13   int tube[2];
14
15   pipe(tube); //creation du tube anonyme
16
```

```
17    child = fork(); //creation de l'enfant
18
19    if( 0 == child){ //code de l'enfant
20      char *buffer  = NULL;
21      size_t taille = 0;
22
23      close(tube[1]); //enfant lecteur = fermeture
24                      //de l'extremite en ecriture
25
26      // Protocole : lecture taille/allocation/lecture donnees
27      read(tube[0],&taille,sizeof(size_t));
28      buffer = malloc(taille*sizeof(char));
29      read(tube[0],buffer,taille); //lecture depuis le tube
30
31      fprintf(stdout,"Message recu = %s\n", buffer);
32
33      free(buffer);
34
35      close(tube[0]); //fermeture de l'extremite en lecture
36
37    } else if ( child > 0) { //code du parent
38      char *buffer  = "Coucou";
39      size_t taille = strlen(buffer)+1;
40
41      close(tube[0]); //parent ecrivain = fermeture
42                      //de l'extremite en lecture
43
44      // Protocole : ecriture taille puis ecriture donnees
45      write(tube[1], &taille, sizeof(size_t));
46      write(tube[1], buffer, strlen(buffer)+1); // ecriture dans
47                                                // le tube
48      close(tube[1]); //fermeture de l'extremite en ecriture
49
50      waitpid(child,NULL,0); // synchronisation avec l'enfant
51    }
52
53    exit(EXIT_SUCCESS);
54  }
```

Correction 2

Voici une seconde correction, un peu plus évoluée. Dans cette version, le processus parent va créér autant d'enfants qu'il souhaite (ce nombre est le premier argument du programme). Ensuite, ce processus parent va communiquer avec ses enfants par le biais du *même* tube. C'est donc une situation avec un écrivain unique et de multiples lecteurs. Ici c'est le même message (`"Coucou"`) qui est envoyé à tous les enfants, mais il serait possible d'envoyer des messages différents (et de taille différente) à chaque enfant. Afin de savoir quelle quantité de données un enfant est censé récupérer exactement, on utilise un second tube dans lequel on va envoyer la taille de la chaîne.

Ainsi l'enfant récupère d'abord la taille puis utilise cette taille pour effectuer la lecture dans le premier tube. Notez qu'il n'y a pas de risque que des données se mélangent puisqu'il n'y a qu'un seul écrivain.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6  #include <sys/stat.h>
7  #include <fcntl.h>
8  #include <assert.h>
9
10 int main(int argc, char *argv[])
11 {
12   assert(argc == 2);
13
14   int    num_fils = atoi(argv[1]);
15   pid_t *fils     = malloc(num_fils*sizeof(pid_t));
16   int    tube[2];
17   int    tube2[2];
18   int    idx;
19
20   pipe(tube);  // creation des deux tubes
21   pipe(tube2); // anonymes
22
23   //creation des enfants
24   for(idx = 0; idx < num_fils ; idx++){
25     fils[idx] = fork();
26
27     if(0 == fils[idx])
28       break;
29     else
30       fprintf(stdout,"creation fils : %i\n",fils[idx]);
31     }
32
33   if(idx < num_fils) //code des enfants
34     {
35       char  *str  = NULL;
36       size_t len  = 0;
37
38       close(tube[1]);  //enfants lecteurs
39       close(tube2[1]); //sur les tubes
40
41       read(tube[0], &len, sizeof(size_t)); // recuperation de
            la taille
42       fprintf(stdout,"[%i] Got msg of len : %li\n", getpid(),
            len);
43
```

```
44        str = malloc(len*sizeof(char)); //allocation du buffer
              pour reception
45
46        read(tube2[0], str, len); //lecture de la chaine
47        fprintf(stdout,"[%i] Got msg %s\n", getpid(), str);
48
49        close(tube[0]); // fermeture des tubes
50        close(tube2[0]);
51
52        free(str); //desallocation du buffer
53      }
54    else if (num_fils == idx) //code du parent
55      {
56        close(tube[0]); //parent ecrivain
57        close(tube2[0]); // sur les deux tubes
58
59        //ecritures dans les tubes
60        for(idx = 0; idx < num_fils ; idx++){
61    char    str[] = "Coucou";
62    size_t len    = strlen(str) + 1;
63
64    write(tube[1], &len, sizeof(size_t));
65    write(tube2[1], str, len);
66        }
67
68        close(tube[1]); //fermeture des tubes
69        close(tube2[1]);
70
71        //synchronisation avec les enfants
72        for(idx = 0; idx < num_fils ; idx++)
73    wait(NULL);
74
75        free(fils);
76      }
77
78    exit(EXIT_SUCCESS);
79  }
```

Exercice 38 ($\star$)

Reprenez l'exercice précédent, mais remplacez le tube *anonyme* par un tube *nommé*.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
```

```c
#include <sys/wait.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <errno.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  // creation du tube
  mkfifo("/tmp/montube",S_IRUSR|S_IWUSR);

  //creation de l'enfant
  pid_t child = fork();

  if( 0 == child ){ //code de l'enfant
    char  *buffer = NULL;
    size_t taille = 0;

    int fd = open("/tmp/montube",O_RDONLY);

    //lecture depuis le tube : taille puis donnees
    read(fd, &taille, sizeof(size_t));
    buffer = malloc(taille*sizeof(char));
    read(fd, buffer, taille);

    fprintf(stdout,"Message recu = %s\n", buffer);

    free(buffer);
    close(fd);

  } else if ( child > 0 ){ //code du parent
    char  *buffer = "Bonjour";
    size_t taille = strlen(buffer) + 1;

    int fd = open("/tmp/montube",O_WRONLY);

    // ecriture dans le tube : taille + donnees
    write(fd, &taille, sizeof(size_t));
    write(fd, buffer, taille);

    close(fd);

    waitpid(child, NULL, 0); // synchronisation avec l'enfant

    unlink("/tmp/montube"); // effacement du fichier
  }

  exit(EXIT_SUCCESS);
}
```

Exercice 39 (⋆⋆)

Écrivez un programme qui effectue l'équivalent de la commande shell suivante :

```
cat toto.txt | grep "truc" | wc -l
```

Indications :
- il n'est **pas** demandé de réécrire les commandes `cat`, `grep` et `wc`.
- l'utilisation des fonctions `system` ou `popen` est interdit.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9
10 #if VERSION_GROUPE == 1
11 int main(int argc, char *argv[])
12 {
13   int index = 0;
14   int tube1[2];
15   int tube2[2];
16
17   pipe(tube1);
18   pipe(tube2);
19
20   for(index  = 0 ; index < 3 ; index++){
21     pid_t child = fork();
22     if(0 == child)
23       break;
24   }
25
26   if(0 == index){ /* enfant #0 */
27     close(tube1[0]);
28     close(tube2[0]);
29
30     close(tube2[1]);
31
32     /* redirection de stdout */
33     close(STDOUT_FILENO);
34     dup(tube1[1]);
35     close(tube1[1]);
36
37     execlp("cat","cat","toto.txt",NULL);
38
39   } else if (1 == index){ /* enfant #1 */
```

```
40      close(tube1[1]);
41      close(tube2[0]);
42
43      close(STDIN_FILENO);
44      dup(tube1[0]);
45      close(tube1[0]);
46
47      close(STDOUT_FILENO);
48      dup(tube2[1]);
49      close(tube2[1]);
50
51      execlp("grep","grep","truc", NULL);
52
53    } else if (2 == index){ /* enfant #2 */
54      close(tube1[0]);
55
56      close(tube1[1]);
57      close(tube2[1]);
58
59      close(STDIN_FILENO);
60      dup(tube2[0]);
61      close(tube2[0]);
62
63      execlp("wc","wc","-l",NULL);
64
65    } else if (3 == index){ /* parent */
66      close(tube1[0]);
67      close(tube1[1]);
68      close(tube2[0]);
69      close(tube2[1]);
70
71      for(index  = 0 ; index < 3 ; index++)
72        wait(NULL);
73    }
74
75    exit(EXIT_SUCCESS);
76  }
77  #else //VERSION_GROUPE == 2
78  int main(int argc, char *argv[])
79  {
80    pid_t child = -1;
81    int tube1[2];
82
83    pipe(tube1);
84
85    child = fork();
86
87    if(child > 0){ /* code parent */
88      close(tube1[1]); /* lecteur uniquement */
```

```
89
90      /* redirection stdin */
91      close(STDIN_FILENO);
92      dup(tube1[0]);
93      close(tube1[0]);
94
95      /* execution de la commande finale */
96      execlp("wc", "wc", "-l", NULL);
97
98    } else if(0 == child){ /* code de l'enfant */
99      int   tube2[2];
100     pid_t child2 = -1;
101
102     close(tube1[0]); /* ecrivain sur tube1 */
103
104     pipe(tube2);
105
106     child2 = fork();
107
108     if(child2 > 0){ /* parent */
109       close(tube2[1]);
110
111       /* redirection stdout */
112       close(STDOUT_FILENO);
113       dup(tube1[1]);
114       close(tube1[1]);
115
116       /* redirection stdin */
117       close(STDIN_FILENO);
118       dup(tube2[0]);
119       close(tube2[0]);
120
121       execlp("grep", "grep", "truc", NULL);
122
123     } else if ( 0 == child2) { /* enfant 2 */
124       close(tube1[1]);
125       close(tube2[0]);
126
127       /* redirection stdout */
128       close(STDOUT_FILENO);
129       dup(tube2[1]);
130       close(tube2[1]);
131
132       execlp("cat","cat", "toto.txt", NULL );
133     }
134   }
135   exit(EXIT_SUCCESS);
136 }
137 #endif
```

4.2 Mémoire partagée

Exercice 40 (⋆⋆)
Créez un programme avec deux processus qui se partagent de la mémoire pour s'échanger des données (une chaîne de caractères par exemple) en utilisant un fichier en support.

Correction

Il faut faire attention à bien synchroniser la lecture avec l'écriture pour la première ne se passe qu'une fois que la seconde a eu lieu. Normalement, il faudrait utiliser un outil de synchronisation comme ceux listés dans le dernier chapitre (et plus particulièrement un sémaphore).

Synchronisation possible avec :

— une ouverture d'un tube nommé ;
— lecture/écriture bloquante dans un tube quelconque
— utilisation d'une variable sentinelle en mémoire partagée indiquant que l'écriture a bien été réalisée. Mais dans ce cas, il faut mettre une barrière-mémoire entre la copie de la chaîne et l'écriture de la sentinelle. Barrières possibles : `__sync_synchronize();` (gcc built-in) ou `atomic_thread_fence` (portable) ou bien qualifier la sentinelle en _Atomic, car cela impose une consistence mémoire aussi.

Côté lecteur :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5  #include <sys/mman.h>
6  #include <string.h>
7  #include <signal.h>
8  #include <errno.h>
9  #include <sys/types.h>
10 #include <sys/stat.h>
11 #include <fcntl.h>
12 #include <assert.h>
13
14 int main(int argc, char **argv)
15 {
16   assert(3 == argc);
17   char  *base_addr = NULL;
18   char  *curr_addr = NULL;
19   char  *filename  = argv[1];
20   size_t map_size  = sysconf(_SC_PAGE_SIZE);
21   int    fd;
22   pid_t  pid;
23
24   pid = getpid();
25   fprintf(stdout,"[%i] === Reader launched\n", pid);
26
27   fd = open(filename, O_RDWR);
28   base_addr = (char *)mmap(NULL, map_size,
29          PROT_READ | PROT_WRITE,
30          MAP_SHARED , fd , 0);
```

```
31    close(fd);
32
33    curr_addr = base_addr + sizeof(int);
34
35    //while( *(volatile _Atomic int *)base_addr != 666)
36    // si utilisation d'une variable atomique
37    while( *(volatile int*)base_addr != 666)
38      fprintf(stdout,"[%i] === Reader in active wait, %i\n",
39        pid, *(volatile int*)base_addr);
40
41    fprintf(stdout,"[%i] === Reader string : %s\n", pid,
        curr_addr);
42    fflush(stdout);
43
44    munmap((void *)base_addr, map_size);
45    return 0;
46  }
```

Côté écrivain :

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <semaphore.h>
4   #include <unistd.h>
5   #include <sys/mman.h>
6   #include <string.h>
7   #include <signal.h>
8   #include <errno.h>
9   #include <sys/types.h>
10  #include <sys/stat.h>
11  #include <fcntl.h>
12  #include <stdatomic.h>
13  #include <assert.h>
14
15  int main(int argc, char **argv)
16  {
17    assert(3 == argc);
18
19    char  *base_addr = NULL;
20    char  *curr_addr = NULL;
21    char  *filename  = argv[1];
22    size_t map_size  = sysconf(_SC_PAGE_SIZE);
23    int    fd;
24
25    fprintf(stdout,"[%i] === Writer launched\n",getpid());
26
27    fd = open(filename,O_RDWR);
28    base_addr = (char *)mmap(NULL, map_size ,
29           PROT_READ | PROT_WRITE,
30           MAP_SHARED , fd , 0);
31    close(fd);
```

```
32
33    memset(base_addr,0,map_size);
34
35
36    curr_addr = base_addr + sizeof(int);
37    memcpy(curr_addr, argv[2], strlen(argv[2])+1);
38
39    // Solutions with explicit barrier:
40
41    //1- __sync_synchronize();
42    // GCC built-in memory barrier (CPU and Compiler).
43    // Not portable.
44
45    //2-  Portable Memory barrier (since C11) :
46    atomic_thread_fence(memory_order_seq_cst);
47    // Note: Sequential consistency (seq_cst) is
48    //       the default memory order in C
49    *( volatile int *)base_addr = 666;
50
51    //3-  Solution with implicit barrier:
52    //*(volatile _Atomic int *)base_addr = 666;
53    //_Atomic (C11) enforces portably
54    // memory (sequential) consistency
55
56    munmap((void *)base_addr,map_size);
57    return 0;
58 }
```

Lanceur :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <signal.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <sys/wait.h>
8  #include <sys/types.h>
9  #include <sys/stat.h>
10 #include <fcntl.h>
11 #include <assert.h>
12
13 int main(int argc, char **argv)
14 {
15   assert( 3 == argc);
16
17   pid_t  pid, pid2;
18   int    fd;
19   size_t map_size = (size_t)sysconf(_SC_PAGE_SIZE);
20
21   fd = open(argv[1],O_RDWR|O_CREAT|O_EXCL,S_IRUSR|S_IWUSR);
```

```
22    if (fd != -1) ftruncate(fd,map_size);
23    close(fd);
24
25    pid = fork();
26
27    if(pid > 0){
28      pid2 = fork();
29      if (pid2 == 0){
30        execlp("./reader","reader",
31         argv[1],argv[2],NULL);
32      } else if (pid2 > 0){
33        waitpid(pid,NULL,0);
34        waitpid(pid2,NULL,0);
35      }
36    } else if (pid == 0){
37      execlp("./writer","writer",argv[1],argv[2],NULL);
38    }
39
40    unlink(argv[1]);
41    exit(EXIT_SUCCESS);
42  }
```

Exercice 41 ($\star$)

Reprenez l'exercice précédent, mais créez des processus **affiliés** et utilisez une projection **anonyme** pour mettre en place la zone de mémoire partagée.

Correction

Synchronisation avec un tube nommé. Problème : système (plus ou moins) one-shot.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <sys/wait.h>
8  #include <sys/types.h>
9  #include <sys/stat.h>
10 #include <fcntl.h>
11 #include <sys/mman.h>
12 #include <assert.h>
13
14 int main(int argc, char **argv)
15 {
16   assert( 2 == argc);
17
18   char  *base_addr = NULL;
19   pid_t  pid;
20   int    fd;
21   size_t map_size = (size_t)sysconf(_SC_PAGE_SIZE);
```

```
22
23    //tube pour la synchronisation
24    mkfifo("/tmp/montube", S_IRUSR | S_IWUSR);
25
26    //creation de la zone de memoire partagee
27    base_addr = (char *)mmap(NULL, map_size ,
28           PROT_READ  | PROT_WRITE,
29           MAP_SHARED | MAP_ANONYMOUS,
30           -1, 0);
31    pid = fork(); // creation du processus-enfant
32
33    if(pid > 0){ //Code du parent
34      size_t str_len = strlen(argv[1]) + 1;
35
36      //Copie en memoire partagee
37      memcpy(base_addr, argv[1], str_len );
38
39      //synchronisation
40      fd = open("/tmp/montube",O_WRONLY);
41
42      //attente terminaison de l'enfant
43      waitpid(pid,NULL,0);
44
45      close(fd);
46      unlink("/tmp/montube");
47
48    } else if (0 == pid){ // Code de l'enfant
49
50      // synchronisation
51      fd = open("/tmp/montube",O_RDONLY);
52
53      //Lecture en memoire partagee
54      fprintf(stdout,"String is : %s\n", base_addr);
55
56      close(fd);
57    }
58
59    munmap((void *)base_addr, map_size);
60    return 0;
61  }
```

Meilleure solution : synchronisation avec des lectures/écritures dans un tube bloquant

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5  #include <sys/mman.h>
6  #include <string.h>
7  #include <signal.h>
8  #include <errno.h>
```

```c
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <assert.h>

int main(int argc, char *argv[])
{
  assert(2 == argc);

  char  *base_addr = NULL;
  char  *curr_addr = NULL;
  size_t map_size  = sysconf(_SC_PAGE_SIZE);
  int    str_len   = strlen(argv[1])+1;
  int    tube[2];

  base_addr = (char *)mmap(NULL, map_size ,
         PROT_READ | PROT_WRITE,
         MAP_SHARED | MAP_ANONYMOUS,
         -1 , 0);
  assert(base_addr != MAP_FAILED);

  curr_addr = base_addr;

  pipe(tube);

  pid_t child = fork();

  if(0 == child){
    char c = 0;

    close(tube[1]);

    read(tube[0],&c, sizeof(char)); //synchro
    fprintf(stdout,"Lecture = %s\n", curr_addr);

    close(tube[0]);

  } else if (child > 0){
    char c = 'a';

    close(tube[0]);

    memset(curr_addr,0, map_size);

    memcpy(curr_addr,argv[1],str_len+1);
    write(tube[1], &c, sizeof(char)); //synchro

    close(tube[1]);
```

```
58      waitpid(child, NULL, 0);
59    }
60
61    munmap((void *)base_addr,map_size);
62    return 0;
63  }
```

Exercice 42 (⋆⋆⋆)
Écrivez un programme qui calcule le minimum d'un tableau d'entiers.
Ce tableau sera partagé par plusieurs processus et sera placé dans une zone de mémoire partagée.

Correction

Utilisation de mémoire partagée pour stocker le tableau et de tubes pour que le parent communique avec ses enfants les paramètres de calcul. Cette solution utilise une variable globale pour stocker le minimum. L'accès à cette variable est protégée par une synchronisation.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <unistd.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <sys/mman.h>
9  #include <assert.h>
10
11 typedef struct son{
12   pid_t pid;
13   int num_elems;
14   int offset;
15 } son_t;
16
17 typedef int pipe_t[2];
18
19 static int syncro[2];
20
21 static int init_syncro(void){
22   char go = 'a';
23   pipe(syncro);
24   write(syncro[1],&go,1);
25   return 0;
26 }
27
28 static void lock(void){
29   char res;
30   read(syncro[0],&res,1);
31 }
32
33 static void unlock(void){
34   char go = 'b';
```

```
35    write(syncro[1],&go,1);
36  }
37
38  static int send_all(int fd, void *buffer, size_t size){
39    ssize_t ret = 0;
40    do{
41      ret += write(fd,(char *)buffer+ret,size-ret);
42    } while(ret != size);
43    return 0;
44  }
45
46  static int recv_all(int fd, void *buffer, size_t size){
47    ssize_t ret = 0;
48    do{
49      ret += read(fd,(char *)buffer+ret,size-ret);
50    } while(ret != size);
51    return 0;
52  }
53
54  static int find_min(volatile int *first,int size)
55  {
56    int idx;
57    int min = *first;
58
59    for(idx = 1; idx < size ; idx++){
60      if(*(first + idx) < min)
61        min = *(first + idx);
62    }
63
64    return min;
65  }
66
67  int main(int argc, char *argv[]){
68
69    assert(3 == argc);
70
71    int num_fils = atoi(argv[1]);
72    int num_vals = atoi(argv[2]);
73    pipe_t *dest = NULL;
74    son_t  *sons = NULL;
75    int     idx;
76    void    *base_ptr = NULL;
77
78    volatile int *array      = NULL;
79    volatile int *global_min = NULL;
80
81    dest = (pipe_t *)malloc(num_fils*sizeof(pipe_t));
82    sons = (son_t  *)malloc(num_fils*sizeof(son_t));
83    memset(sons,0,num_fils*sizeof(son_t));
```

```
84
85    base_ptr = mmap(NULL, (num_vals + 1)*sizeof(int),
86        PROT_READ | PROT_WRITE,
87        MAP_SHARED | MAP_ANONYMOUS, -1, 0);
88    assert(base_ptr != MAP_FAILED);
89
90    global_min = (volatile int *)base_ptr;
91    array = ++global_min;
92
93    fprintf(stdout,"Array start @%p\n",array);
94
95    srand(getpid());
96    for(idx = 0; idx < num_vals; idx++){
97      array[idx] = rand();
98      fprintf(stdout,"Master[%2i]:%12i\n",idx,array[idx]);
99    }
100
101   *global_min= array[0];
102
103   init_syncro();
104
105   for(idx = 0; idx < num_fils; idx++)
106     {
107       sons[idx].num_elems = num_vals/num_fils;
108       if((num_vals%num_fils) && (idx < num_vals%num_fils)){
109   sons[idx].num_elems++;
110       }
111       if (idx > 0){
112   sons[idx].offset =
113     sons[idx-1].num_elems+sons[idx-1].offset;
114       }
115       pipe(dest[idx]);
116
117       sons[idx].pid = fork();
118
119       if(0 == sons[idx].pid){
120   int idx2;
121   int offset;
122   int size;
123   int local_min;
124
125   close(dest[idx][1]);
126   for(idx2 = 0 ; idx2 < idx ; idx2++){
127     close(dest[idx2][1]);
128   }
129
130   recv_all(dest[idx][0],&offset,sizeof(int));
131   recv_all(dest[idx][0],&size,sizeof(int));
132   fprintf(stdout,"Proc[%i] handles %i elements @ %p\n",
```

```
133       getpid(),size,array+offset);
134
135     local_min = find_min(array+offset,size);
136     fprintf(stdout,"Proc[%i] found local min : %i\n",
137       getpid(),local_min);
138
139     lock();
140     if (local_min < *global_min)
141       *global_min = local_min;
142     unlock();
143
144     exit(EXIT_SUCCESS);
145         } else if(sons[idx].pid > 0){
146     close(dest[idx][0]);
147         }
148       }
149
150     for(idx = 0; idx < num_fils; idx++){
151       send_all(dest[idx][1],&sons[idx].offset,sizeof(int));
152       send_all(dest[idx][1],&sons[idx].num_elems,sizeof(int));
153     }
154
155     fprintf(stdout,"=============> Global min is %i (probably
          false)\n",*global_min);
156
157     for(idx = 0; idx < num_fils; idx++){
158       wait(NULL);
159     }
160
161     fprintf(stdout,"=============> Global min is
          %i\n",*global_min);
162
163     exit(EXIT_SUCCESS);
164 }
```



5. Interlude non-déterministe



6. Notification d'événements : les signaux

6.1 Gestion des événements par interruption

6.2 Principes de fonctionnement des signaux

Exercice 43 (⋆⋆)
À partir de l'examen des structures utilisées pour l'implémentation des signaux, combien d'occurences de signal d'un type donné peuvent être dans l'état *pendant* dans un processus ?

Correction

L'information de signal pendant étant codée sur 1 bit, une unique occurence d'un signal de type donné est stockée dans un processus. Dans le cas des signaux temps-réels, il est possible de stocker jusqu'à 1024 occurences.

6.3 Émission et réception d'un signal

Exercice 44 (⋆)
Écrivez un programme qui affiche son pid puis qui effectue une boucle infinie. Depuis un terminal (autre que celui qui a servi à lancer le programme initial), envoyez un signal de type `SIGKILL` ou `SIGTERM` à ce processus.

Correction

Code du programme à exécuter :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <unistd.h>
```

```c
#include <signal.h>

int main(int argc, char *argv[])
{
  fprintf(stdout," pid = %i\n",getpid());

  while(1);

  exit(EXIT_SUCCESS);
}
```

Commande dans le terminal : `kill -SIGKILL` *pid du processus*

Exercice 45 ($\star$)

Reprenez l'exercice précédent, mais l'émission du signal doit maintenant se faire par un processus et non avec une commande shell tapée dans un terminal.

Correction

Le processus à signaler a le même code que dans l'exercice précédent. L'envoi du signal se fait dans ce cas avec un appel à `kill` dans un processus.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>

int main(int argc, char *argv[])
{
  // Le programme prend le pid du processus a
  // signaler en argument
  pid_t process_to_signal = atoi(argv[1]);

  kill(process_to_signal, SIGTERM);

  exit(EXIT_SUCCESS);
}
```

6.4 Traitants de signaux

Exercice 46 ($\star$)

Écrivez un programme qui redéfinit son comportement vis-à-vis du signal `SIGUSR1` : à chaque réception d'un tel signal, le processus devra afficher une chaîne de caractères indiquant qu'il a bien reçu ce signal. Vous émettrez ces signaux de type `SIGUSR1` depuis un terminal.

Correction

```c
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/wait.h>
#include <errno.h>
#include <assert.h>
#include <signal.h>

// Etape 1: ecriture du traitant
void mon_joli_traitant(int signum){
  char *str = "Coucou\n";
  // write est une fonction async-signal-safe
  write(STDOUT_FILENO,str,strlen(str));
}

int main(int argc, char *argv[])
{
  fprintf(stdout,"mon pid est %i\n",getpid());

  // Etape 2 : declaration et initialisation
  // d'une structure de type struct sigaction
  struct sigaction action;
  memset(&action,0,sizeof(struct sigaction));
  action.sa_handler = mon_joli_traitant;

  // Etape 3 : appel a la fonction sigaction
  // pour la mise en place effective du traitant
  sigaction(SIGUSR1,&action,NULL);

  while(1);

  exit(EXIT_SUCCESS);
}
```

Exercice 47 (⋆⋆)

Reprenez l'exercice précédent, mais au lieu d'envoyer à ces signaux via une commande shell depuis un terminal, faites-le depuis un second programme C.

— Que constatez-vous ?

— Ce comportement est-il cohérent avec ce que vous connaissez du fonctionnement des signaux ?

Correction

Le code du processus récepteur des signaux est à reprendre de l'exercice précédent. Le code du processus émetteur est le suivant :

```c
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/wait.h>
#include <errno.h>
#include <assert.h>
#include <signal.h>

int main(int argc, char *argv[])
{
  pid_t pid = atoi(argv[1]);
  fprintf(stdout,"mon pid est %i\n",getpid());

  for(int i = 0 ; i < 10 ; i++){
    fprintf(stdout,"Emission du signal au processus :
       %i\n",pid);
    kill(pid, SIGUSR1);
  }

  exit(EXIT_SUCCESS);
}
```

Le processus récepteur ne va pas exécuter 10 fois le traitant de signal car il se passe le phénomène de *rafale de signaux*. C'est donc totalement cohérent avec ce que vous avez vu au sujet des signaux.

Exercice 48 (⋆⋆⋆)
Pourquoi un signal est-il **automatiquement bloqué** lors de l'exécution du traitant mis en place par un processus pour réagir à la réception de ce même signal ?

Correction

Posons-nous la question à l'envers : que se passerait-il si le signal n'était pas *automatiquement* bloqué dans le traitant ? Dans un tel cas, il existerait une possibilité pour se déclenche un **débordement de pile** à cause de l'imbrication du même traitant un grand nombre de fois. Masquer le signal permet de se prémunir d'une telle imbrication et donc évite le problème. Notez cependant qu'il n'est pas interdit d'imbriquer l'exécution de plusieurs traitants. Cependant, cela n'est pas problématique car le nombre d'imbrications sera forcément limité (au maximum le nombre de signaux supportés par le système d'exploitation) : un débordement de pile n'est donc pas possible.

6.4.1 Interactions entre appels-système et traitants de signaux

6.5 Contrôle des signaux avec les masques

Exercice 49 (⋆)

1. Écrivez un programme qui met en place un traitant de signal pour `SIGUSR1` et envoyez ce signal au processus avec une commande depuis un terminal.
2. Ensuite, modifiez le programme pour bloquer `SIGUSR1` et vérifiez que le masque est bien mis en place en continuant à envoyer le signal avec une commande depuis un terminal.

Correction

Cet exercice sera traité de la façon suivante :

1. mise en place du traitant pour `SIGUSR1` ;
2. premier appel à `pause` ;
3. mise en place d'un masque pour bloquer `SIGUSR1` ;
4. second appel à `pause`.

Le signal `SIGUSR1` est envoyé deux fois depuis un terminal et normalement le processus récepteur devrait sortir du premier appel à `pause`, mais pas du second puisqu'un masque a été installé (un seul message « `Je sors de pause` » doit s'afficher). Voilà le code correspondant :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <signal.h>
10
11 void mon_joli_traitant(int signum){
12   char *str = "Je sors de pause\n";
13   write(STDOUT_FILENO,str,strlen(str));
14 }
15
16 int main(int argc, char *argv[])
17 {
18   fprintf(stdout,"mon pid est %i\n",getpid());
19
20   // Mise en place du traitant
21   struct sigaction action;
22   memset(&action,0,sizeof(struct sigaction));
23   action.sa_handler = mon_joli_traitant;
24   sigaction(SIGUSR1,&action,NULL);
25
26   // Premier appel a pause
27   pause();
28
29   // Mise en place du masque
30   sigset_t mask;
31   sigemptyset(&mask);
32   sigaddset(&mask, SIGUSR1);
33   sigprocmask(SIG_BLOCK,&mask,NULL);
34
35   // Second appel a pause
36   pause();
37
38   exit(EXIT_SUCCESS);
39 }
```

Exercice 50 (⋆⋆)

Rappel : un processus ne prend pas forcément en compte les signaux dans l'ordre où ils ont été émis. À l'aide des masques, faites en sorte de forcer cet ordre. Effectuez des tests avec `SIGUSR1`, `SIGUSR2` et `SIGSEGV` et vérifiez que votre programme fonctionne correctement : par exemple vous allez envoyer `SIGSEGV` en premier, mais vous voulez qu'il soit pris en compte en dernier, après `SIGUSR1` et `SIGUSR2`. Vous utiliserez une variable partagée entre le `main` et les traitants pour faire cet exercice.

Correction

Prenons par exemple le processus émetteur dont le code est le suivant :

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
  pid_t pid = atoi(argv[1]);

  kill(pid,SIGSEGV);
  sleep(2);
  kill(pid,SIGUSR1);
  sleep(2);
  kill(pid,SIGUSR2);

  fprintf(stdout,"All signals sent\n");

  exit(EXIT_SUCCESS);
}
```

. Une façon de recevoir les signaux dans l'ordre demandé est illustrée par le code ci-dessous :

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <string.h>
#include <sys/types.h>
#include <unistd.h>

volatile sig_atomic_t signaled = 0;

void handler(int sig)
{
  if(SIGUSR1 == sig){
    write(STDOUT_FILENO,"SIGUSR1 recu\n",14);
    signaled++;
  } else if (SIGUSR2 == sig){
    write(STDOUT_FILENO,"SIGUSR2 recu\n",14);
    signaled++;
```

```
18    } else if (SIGSEGV == sig){
19      write(STDOUT_FILENO,"SIGSEGV recu\n",14);
20    }
21  }
22
23  int main(int argc, char *argv[])
24  {
25    fprintf(stdout,"Mon pid est : %i\n",getpid());
26
27    // Mise en place du masque pour bloquer SIGSEGV
28    sigset_t mask;
29    sigemptyset(&mask);
30    sigaddset(&mask,SIGSEGV);
31    sigprocmask(SIG_BLOCK,&mask,NULL);
32
33    // Mise en place du traitant pour SIGSEGV
34    struct sigaction action;
35    memset(&action,0,sizeof(struct sigaction));
36    action.sa_handler = handler;
37    sigaction(SIGSEGV,&action,NULL);
38
39    // Mise en place du traitant pour SIGUSR1
40    // On bloque automatiquement SIGUSR2 dans son traitant
41    // pour eviter les problemes lors de l'incrementation
42    // de la variable signaled
43    sigaddset(&action.sa_mask,SIGUSR2);
44    sigaction(SIGUSR1,&action,NULL);
45
46    // Mise en place du traitant pour SIGUSR2
47    // On bloque automatiquement SIGUSR1 dans son traitant
48    // (meme raison que ci-dessus)
49    sigaddset(&action.sa_mask,SIGUSR1);
50    sigaction(SIGUSR2,&action,NULL);
51
52    // Attente de SIGUSR1 et SIGUSR2
53    while(signaled != 2){
54      pause();
55    }
56
57    // Deblocage de SIGSEGV
58    sigprocmask(SIG_UNBLOCK,&mask,NULL);
59
60    exit(EXIT_SUCCESS);
61  }
```

6.6 Points Divers

6.7 Travail dirigé : gestion des zombies avec les signaux

6.7.1 Récupération des zombies et traitant de signal

Afin de mettre en pratique la gestion des signaux, je vous propose un travail sur la récupération des enfants zombies d'un processus.

Question 1

Quelles sont les différents moyens/techniques pour récupérer un enfant zombie ?

Question 2

Quels sont les avantages/inconvénients de ces méthodes ?

Question 3

On suppose maintenant que l'on va travailler sur un programme qui répond à la question du premier exercice de la Section 3.7. Quel est le problème posé par les enfants zombies dans le cas d'un tel programme ? Quelle méthode proposez-vous pour la prise en compte de ces zombies ?

Question 4

Écrivez une première version d'un traitant avec la fonction `wait`. Testez-là. fonctionne-t-elle ? Cette version est-elle satisfaisante dans **tous** les cas ?

Question 5

Améliorez votre traitant pour une meilleure prise en compte des zombies en évitant le problème de la rafale. Testez à nouveau votre code : est-il fonctionnel ?

Question 6

Que faut-il modifier pour obtenir une version complètement fonctionnelle ? Indice : allez relire les règles d'écriture d'un traitant (cf. Section **??**).

6.7.2 Et maintenant, du côté du `main` ...

Il n'est pas déraisonnable de penser que l'ensemble des processus-enfants zombies sera pris en compte lors de l'exécution du `main`, c'est-à-dire pendant son travail principal. Il peut cependant arriver que ce ne soit pas toujours le cas et alors, le `main` doit attendre que l'ensemble des zombies soit traité avant de terminer son exécution. Cette attente peut prendre plusieurs formes, mais celle-ci est assez raisonnable :

```
1  sig_atomic_t num_enfants = 0;
2
3  /* traitant pour le signal SIGCHLD */
4
5  void traitant(int sig)
6  {
7    /* code du traitant : cf. partie 1*/
8    /* quand recuperation d'un zombie : */
9
10   num_enfants--;
11 }
12
13 int main(int argc, char **argv)
14 {
15   fd = accept(....);
```

```
16
17    if(fd > 0){
18      /* creation d'un enfant */
19      fork();
20      num_enfants++;
21     }
22
23    /* travail du main */
24
25    /* eventuellement, on peut n'avoir
26       plus rien a faire : attente passive */
27
28    while(num_enfants > 0){
29      pause();
30    }
31
32    exit(EXIT_SUCCESS);
33  }
```

Question 1

La gestion de la variable `num_enfants` est-elle correcte ? Que faut-il modifier/ajouter éventuellement ?

Question 2

L'appel à `pause` est-il obligatoire ? Quelle solution alternative proposez-vous ?

Question 3

En supposant que le traitant correspond à celui obtenu à la fin de la partie 1, la solution d'attente dans le `main` est-elle fonctionnelle dans **tous** les cas ?

Question 4

Écrivez une nouvelle solution qui met en place des masques de signaux. Le problème est-il réglé ?



7. Splendeur et tourments du multithreading

7.1 Notions de multithreading

7.2 L' interface Pthread

Exercice 51 ($\star$)
Écrivez un programme qui crée deux threads (en plus du `main`). Chaque thread exécutera sa propre fonction. Le `main` effectuera une boucle d'attente active ou un `sleep` de quelques secondes avant de se terminer.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <pthread.h>
10
11 #define NTHREADS (2)
12
13 void *f1(void *arg){
14   fprintf(stdout,"Coucou\n");
15   return NULL;
16 }
17
18 void *f2(void *arg){
19   fprintf(stdout,"Hello\n");
```

```
20    return NULL;
21  }
22
23  int main(int argc, char *argv[])
24  {
25    pthread_t tids[NTHREADS];
26    void *(*f_ptr_array[])(void *) = {f1,f2};
27
28    for(int i = 0 ; i < NTHREADS ; i++){
29      pthread_create(tids+i,NULL,f_ptr_array[i%2], NULL);
30    }
31
32    sleep(5);
33
34    exit(EXIT_SUCCESS);
35  }
```

Exercice 52 ($\star$)

Modifiez le programme précédent afin que les deux threads exécutent la même fonction, mais avec un paramètre différent.

Correction

D'après la section 6.3.2.3 du standard C11 (paragraphes #5 et #6), il n'est pas portable de convertir un entier en pointeur et de reconvertir ce pointeur à nouveau en entier. Il n'est pas garanti que la valeur obtenue soit égale à la valeur d'origine (i.e. le test `int(void *)val == val` n'est pas toujours vrai). Il faut donc passer un pointeur sur l'argument à passer au thread, puis transtyperce pointeur dans le bon type (aui est connu) dans le thread pour récupérer l'argument.

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <unistd.h>
4   #include <fcntl.h>
5   #include <string.h>
6   #include <sys/wait.h>
7   #include <errno.h>
8   #include <assert.h>
9   #include <pthread.h>
10
11  #define NTHREADS (2)
12
13  void *thread_func(void *arg){
14
15    // Conversion du pointeur generique en pointeur sur entier
16    // puis dereferencement de ce pointeur pour recuperer
17    // l'argument
18    int val = *(int *)arg;
19    fprintf(stdout,"Coucou === %i\n",val);
20    return NULL;
21  }
22
```

```
23 int main(int argc, char *argv[])
24 {
25   pthread_t tids[NTHREADS];
26   int args[NTHREADS] = {111,222};
27
28   for(int i = 0 ; i < NTHREADS ; i++){
29     // Passage de l'adresse de l'argument
30     // Cette adresse est convertie en pointeur generique.
31     pthread_create(tids+i,NULL,
32        thread_func,
33        (void *)&args[i]);
34   }
35
36   sleep(5);
37   exit(EXIT_SUCCESS);
38 }
```

Exercice 53 (⋆⋆)

Écrivez un programme qui crée 10 threads. Chaque thread devra afficher son « indice » de création. Le `main` effectuera une boucle d'attente active ou un `sleep` de quelques secondes avant de se terminer.

— Est-ce conforme à vos attentes ?

— Comment expliquez-vous ce que vous observez ?

Correction

Une première correction pourrait être la suivante :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <pthread.h>
10
11 #define NTHREADS (10)
12
13 void *f(void *arg){
14   int val = *(int*)arg;
15   fprintf(stdout,"Coucou from thread %i\n",val);
16   return NULL;
17 }
18
19 int main(int argc, char *argv[])
20 {
21   pthread_t tids[NTHREADS];
```

```
22
23    for(int i = 0 ; i < NTHREADS ; i++){
24      pthread_create(tids+i,NULL,f, (void *)&i);
25    }
26
27    sleep(5);
28    exit(EXIT_SUCCESS);
29  }
```

Avec cette version, on voit que l'on n'obtient pas la séquence des dix nombres de 0 à 9. Il arrive même que 10 soit affiché (notez que le fait que les nombres ne s'affichent pas dans un ordre croissant n'est — en revanche — pas un problème). Le souci vient du fait que l'on donne en argument *l'adresse* de la variable `i` et qu'il existe un décalage entre la valeur de `i` au moment de la création du thread et le début de son exécution. La variable est modifiée par le thread principal (celui qui exécute la fonction `main`) et cette modification est visible dans les threads. Donc, une version correcte pourrait être la suivante :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <pthread.h>
10
11 #define NTHREADS (10)
12
13 void *f(void *arg){
14   int val = *(int*)arg;
15   fprintf(stdout,"Coucou from thread %i\n",val);
16   return NULL;
17 }
18
19 int main(int argc, char *argv[])
20 {
21   pthread_t tids[NTHREADS];
22   int args[NTHREADS] = {0};
23
24   for(int i = 0 ; i < NTHREADS ; i++){
25     args[i] = i;
26     pthread_create(tids+i,NULL,f, (void *)&args[i]);
27   }
28
29   sleep(5);
30   exit(EXIT_SUCCESS);
31 }
```

Exercice 54 (⋆)
Comparez le prototype d'une fonction exécutée par un thread avec celui d'un traitant de signal : quels commentaires pouvez-vous formuler ?

Correction

Un thread exécute une fonction avec un prototype bien plus flexible et générique que celui d'un traitant de signal. Par exemple, un thread a la possibilité de renvoyer quelque chose, ce qui n'est pas le cas d'un traitant de signal.

Exercice 55 (⋆)
Quel(s) moyen(s) pouvez-vous proposer pour passer plus d'un argument à une fonction exécutée par un thread ?

Correction

Pour passer plusieurs arguments, il suffit de créer une structure ayant pour champs l'ensemble des arguments que l'on veut passer au thread et on lui passe un pointeur sur cette structure. Par exemple, supposons qu'on veuille faire exécuter à un thread une fonction `f` dont le prototype est le suivant : `void *f(int arg1, char *arg2, int arg3);` On va alors déclarer une structure de ce type :

```
1  typedef struct my_args{
2    int   arg1;
3    char *arg2;
4    int   arg3;
5  } my_args_t;}
```

Puis on écrit une fonction g qui va juste récupérer les arguments et les passer à la fonction `f` :

```
1   void *g(void *arg){
2
3     my_args_t *args_ptr = (my_args_t *)arg;
4
5     void *res = f(args_ptr->arg1,
6       args_ptr->arg2,
7       args_ptr->arg3);
8
9     pthread_exit(res);
10 }
```

Et au lieu de faire exécuter la fonction `f` au thread, on lui fait exécuter la fonction g, qui a le bon prototype (ie un seul argument, de type `void *`).

```
1  pthread_t tid;
2  my_args_t args = {.arg1 = ...,
3                    .arg2 = ...,
4                    .arg3 = ...};
5
6  pthread_create(&tid, NULL, g, (void *)&args);
```

Exercice 56 (⋆)
Écrivez un programme qui crée quelques threads. Ces threads afficheront un message avant

de se mettre en sommeil[a]. Le thread exécutant le `main` appellera `pthread_exit` une fois ces créations effectuées en lieu et place de la fonction `exit`.
— Que constatez-vous ?
— Est-ce conforme à vos attentes ?

a. La fonction `sleep` permet de mettre en sommeil le thread appelant, pas tout le processus ...

Correction

Le thread principal est terminé mais pas tout le processus. Les autres threads continuent leur exécution.

Exercice 57 (⋆)

Reprenez les programmes précédents et enlevez les temporisations (i.e. les boucles d'attente active ou les appels à `sleep`) dans la fonction `main`.
— Que constatez-vous ?
— Comment corriger le problème ?

Correction

On peut constater des problèmes de synchronisation/exécution des programmes. Par exemple, dans certains exercices précédents, si le thread principal ne temporise pas, il appelle `exit` et termine tous le processus avant même que les autres threads aient pu commencer leur exécution. Il faut donc synchroniser les exécutions des différents threads avec des appels à la fonction `pthread_join`.

Exercice 58 (⋆⋆)

Écrivez un programme dont le `main` :
1. crée quelques threads ;
2. attend leur terminaison ;
3. récupère les codes de retour et en fait la somme (on supposera que le résultat attendu de chaque thread est un entier).

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <pthread.h>
10
11 #define NTHREADS (10)
12
13 void *thread_func(void *arg){
14   int indice = *(int *)arg;
15   fprintf(stdout,"Thread === %i\n",indice);
16
17   int *resultat = malloc(sizeof(int));
18   *resultat = 1;
19
```

```
20    for(int i = 0; i < indice ; i++){
21      (*resultat) *= 10;
22    }
23
24    pthread_exit((void *)resultat);
25  }
26
27  int main(int argc, char *argv[])
28  {
29    pthread_t tids[NTHREADS];
30    int       indices[NTHREADS] = {0};
31    int      *results[NTHREADS] = {NULL};
32    int       result = 0;
33
34    for(int i = 0 ; i < NTHREADS ; i++){
35      indices[i] = i;
36      pthread_create(tids+i,NULL,
37         thread_func,
38         (void *)(indices+i));
39    }
40
41    for(int i = 0 ; i < NTHREADS ; i++){
42      pthread_join(tids[i],(void **)&(results[i]));
43      result += *(results[i]);
44    }
45
46    fprintf(stdout,"Resultat = %i\n",result);
47
48    exit(EXIT_SUCCESS);
49  }
```

Exercice 59 (⋆⋆)

1. Écrivez un programme qui calcule le minimum d'un tableau de m entiers (les valeurs pourront être générées aléatoirement).
2. Mesurez le temps pris par ce programme pour plusieurs valeurs de m.
3. Modifiez ensuite ce programme pour que le calcul soit réparti parmi n threads où chaque thread devra donc traiter un sous-tableau d'environ m/n entiers (m n'est pas forcément un multiple de n). Refaites les mesures de temps pour plusieurs valeurs de m et n (1, 2, 4, etc.). Que constatez-vous ?

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <pthread.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <stdint.h>
```

```
8  #include <inttypes.h>
9  #include <semaphore.h>
10 #include <sys/time.h>
11 #include <string.h>
12 #include <assert.h>
13
14 #define M 1000000
15
16 typedef struct th_arg{
17   int64_t *base_ptr;
18   int64_t num_elems;
19 } th_arg_t;
20
21
22 void *sort(void *arg)
23 {
24   int64_t *ptr = ((th_arg_t *)arg)->base_ptr;
25   int64_t nelems = ((th_arg_t *)arg)->num_elems;
26   int64_t min = *ptr;
27   struct timeval t1,t2;
28   double time = 0.0;
29
30   gettimeofday(&t1,NULL);
31   for(int64_t i = 1 ; i < nelems ; i++)
32     if( ptr[i] < min)
33       min = ptr[i];
34   gettimeofday(&t2,NULL);
35
36   time = (t2.tv_sec - t1.tv_sec) + (t2.tv_usec -
       t1.tv_usec)/1000000.0;
37   fprintf(stdout,"Time spent in subsort: %f sec \n",time);
38
39   pthread_exit((void *)min);
40 }
41
42 int main(int argc, char *argv[])
43 {
44   int64_t   *garray = NULL;
45   int64_t   *marray = NULL;
46   pthread_t *tid = NULL;
47   th_arg_t  *args = NULL;
48   int num_threads = 10;
49   int64_t  min;
50   struct timeval t1,t2;
51   float time = 0.0;
52   int seed = 666;
53   memset(&t1,0,sizeof(struct timeval));
54   memset(&t2,0,sizeof(struct timeval));
55
```

```
56    if(3 == argc) {
57      num_threads = atoi(argv[1]);
58      seed = atoi(argv[2]);
59    }
60
61    fprintf(stdout,"creating %i threads\n",num_threads);
62    srand(seed);
63
64    tid =  (pthread_t *)malloc(num_threads * sizeof(pthread_t));
65    args =  (th_arg_t *)malloc(num_threads * sizeof(th_arg_t));
66    marray = (int64_t *)malloc(num_threads * sizeof(int64_t));
67    garray = (int64_t *)malloc(M * sizeof(int64_t));
68
69    for(int64_t i = 0 ; i < M ; i++)
70      garray[i] = rand();
71
72    gettimeofday(&t1,NULL);
73    for(int i = 0 ; i < num_threads ; i++){
74      args[i].base_ptr  = garray + i*(M/num_threads);
75      args[i].num_elems = M/num_threads;
76    }
77    args[num_threads-1].num_elems += M%num_threads;
78
79    for(int i = 0 ; i < num_threads ; i++)
80      pthread_create(&tid[i],NULL,sort,(void *)&(args[i]));
81
82    for(int i = 0 ; i < num_threads ; i++)
83      pthread_join(tid[i],(void *)&marray[i]);
84
85    gettimeofday(&t2,NULL);
86
87    min = marray[0];
88    for(int i = 1 ; i < num_threads ; i++)
89      if(marray[i] < min)
90        min = marray[i];
91
92    time  = (t2.tv_sec - t1.tv_sec) + (t2.tv_usec -
         t1.tv_usec)/1000000.0;
93    fprintf(stdout,"Time spent: %f sec\n",time);
94
95    free(tid);
96    free(args);
97    free(marray);
98    free(garray);
99
100   fprintf(stdout,"End of main: Min  is %" PRId64 "\n",min);
101
102   return 0;
103 }
```

7.3 Concepts avancés

Exercice 60 (⋆)
Écrivez un programme qui crée quelques threads utilisant une variable stockées dans leur TLS respectifs et affichez la valeur de cette dernière. Est-il possible de se passer de ce mécanisme ?

Correction

Exemple de déclaration et d'utilisation d'une variable en TLS. Le nom de la variable est bien global mais sa valeur est locale à chaque thread.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <pthread.h>
10
11 #define NTHREADS (10)
12
13 _Thread_local int variable = 0;
14
15 void *thread_func(void *arg){
16   int indice = *(int *)arg;
17   variable += indice;
18   fprintf(stdout,"Thread[%i] === %i\n",indice,variable);
19   pthread_exit(NULL);
20 }
21
22 int main(int argc, char *argv[])
23 {
24   pthread_t tids[NTHREADS];
25   int       indices[NTHREADS] = {0};
26
27   for(int i = 0 ; i < NTHREADS ; i++){
28     indices[i] = i;
29     pthread_create(tids+i,NULL,
30        thread_func,
31        (void *)(indices+i));
32   }
33
34   for(int i = 0 ; i < NTHREADS ; i++){
35     pthread_join(tids[i],NULL);
36   }
37
38   exit(EXIT_SUCCESS);
39 }
```

Il est souvent possible de se passer de ce mécanisme par le biais des arguments qu'on passe à un thread. Il reste des cas où ce n'est pas toujours faisable (ex : `errno`).

Exercice 61 (⋆⋆)

Écrivez un programme dont le main crée trois threads qui effectuent chacun une boucle infinie et affichent une chaîne de caractères. À l'aide de masques, faites en sorte que le signal `SIGUSR1` soit reçu par le deuxième thread et le signal `SIGUSR2` par le troisième thread. L'émission des signaux se fera avec la commande `kill` depuis un autre terminal.

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <signal.h>
10 #include <pthread.h>
11
12 #define NTHREADS (3)
13
14 void traitant(int sig){
15   if(SIGUSR1 == sig){
16     write(STDOUT_FILENO,"signal SIGUSR1\n",15);
17   } else if (SIGUSR2 == sig){
18     write(STDOUT_FILENO,"signal SIGUSR2\n",15);
19   }
20 }
21
22 void *f1(void *arg){
23   fprintf(stdout,"=== Thread 1\n");
24   pause();
25   fprintf(stdout,"=== Thread 1 sorti de pause\n");
26   pthread_exit(NULL);
27 }
28
29 void *f2(void *arg){
30   fprintf(stdout,"=== Thread 2\n");
31   sigset_t mask;
32   sigemptyset(&mask);
33   sigaddset(&mask,SIGUSR1);
34
35   //On debloque SIGUSR1 pour le thread 2
36   pthread_sigmask(SIG_UNBLOCK, &mask, NULL);
37   pause();
38   fprintf(stdout,"=== Thread 2 sorti de pause\n");
39
```

```
40    pthread_exit(NULL);
41  }
42
43  void *f3(void *arg){
44    fprintf(stdout,"=== Thread 3\n");
45    sigset_t mask;
46    sigemptyset(&mask);
47    sigaddset(&mask,SIGUSR2);
48
49    //On debloque SIGUSR2 pour le thread 3
50    pthread_sigmask(SIG_UNBLOCK, &mask, NULL);
51    pause();
52    fprintf(stdout,"=== Thread 3 sorti de pause\n");
53
54    pthread_exit(NULL);
55  }
56
57
58  int main(int argc, char *argv[])
59  {
60    pthread_t tids[NTHREADS];
61    void *(*f_ptr[])(void *) = {f1, f2, f3};
62
63    fprintf(stdout," === pid = %i\n",getpid());
64
65    //Mise en place des masques pour *tous* les threads
66    sigset_t mask;
67    sigemptyset(&mask);
68    sigaddset(&mask,SIGUSR1);
69    sigaddset(&mask,SIGUSR2);
70    pthread_sigmask(SIG_BLOCK, &mask, NULL);
71
72    // Mise en place des traitants pour tous les threads
73    struct sigaction action = {0};
74    action.sa_handler = traitant;
75    sigaction(SIGUSR1, &action, NULL);
76    sigaction(SIGUSR2, &action, NULL);
77
78    for(int i = 0 ; i < NTHREADS ; i++){
79      pthread_create(tids+i, NULL, f_ptr[i%NTHREADS], NULL);
80    }
81
82    for(int i = 0 ; i < NTHREADS ; i++){
83      pthread_join(tids[i],NULL);
84    }
85
86    exit(EXIT_SUCCESS);
87  }
```



8. Synchronisation inter-flots

8.1 Les sémaphores

Exercice 62 (⋆⋆)

Démarrage synchronisé : écrivez un programme qui lance dix processus-enfants. Chaque enfant se contente d'afficher une chaîne de caractères qui lui est propre, mais cet affichage doit se produire uniquement quand **tous** les enfants ont effectivement été créés par le processus-parent.

— Version 1 : utilisez un sémaphore nommé
— Version 2 : utilisez un sémaphore anonyme

Correction

Il est possible de faire cet exercice en utilisant un sémaphore anonyme ou nommé.

1. Version fonctionnelle avec un sémaphore anonyme. Remarquez bien que le sémaphore est placé dans une zone de mémoire partagée entre les différents processus créés :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5  #include <sys/mman.h>
6  #include <string.h>
7  #include <signal.h>
8  #include <sys/wait.h>
9  #include <assert.h>
10 #include <stdalign.h>
11
12 #define MAP_SIZE (4*1024)
13 #define N (10)
14
15 int main(int argc, char *argv[]){
```

```
16  pid_t  pid_fils[N];
17  sem_t *sem_addr  = NULL;
18  char  *base_addr = NULL;
19  char  *curr_addr = NULL;
20  int    i;
21
22  base_addr = (char *)mmap(NULL, MAP_SIZE ,
23         PROT_READ  | PROT_WRITE,
24         MAP_SHARED | MAP_ANONYMOUS,
25         -1, 0);
26  curr_addr = base_addr;
27
28  //Alignment check + modification
29  size_t offset =
       (size_t)(((intptr_t)curr_addr)%alignof(sem_t));
30  fprintf(stdout,"Semaphore status is : %s",
31    (offset == 0) ? "aligned\n" : "not aligned\n" );
32  curr_addr += offset;
33
34  sem_addr = (sem_t *)curr_addr;
35  sem_init(sem_addr, 1 , 0);
36
37  curr_addr += sizeof(sem_t);
38
39  for(i = 0 ; i < N ; i++){
40    pid_fils[i] = fork();
41    if( 0 == pid_fils[i])
42      break;
43  }
44
45  if (i < N){
46    sem_wait(sem_addr);
47    fprintf(stdout,"Enfant %i OK\n",getpid());
48  } else if (i == N){
49    pid_t pid;
50    sleep(10);
51
52    for(int i = 0; i < N; i++)
53      sem_post(sem_addr);
54
55    for(int i = 0; i < N; i++){
56      pid = wait(NULL);
57      fprintf(stdout,"Enfant %i fini \n",pid);
58    }
59    sem_destroy(sem_addr);
60  }
61
62  munmap((void *)base_addr,MAP_SIZE);
63  exit(EXIT_SUCCESS);
```

```
64 }
```

2. Version fonctionnelle avec un sémaphore nommé. Comme le sémaphore est accessible via son nom (dans le SGF) par tous les processus (à partir du moment où ils connaissent ce nom), ce sémaphore n'a pas besoin d'être placé dans une zone de mémoire partagée :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5  #include <sys/mman.h>
6  #include <string.h>
7  #include <signal.h>
8  #include <sys/wait.h>
9  #include <assert.h>
10 #include <stdalign.h>
11 #include <sys/stat.h>
12 #include <fcntl.h>
13
14 #define N (10)
15
16 int main(int argc, char *argv[])
17 {
18   pid_t  pid_fils[N];
19   sem_t *sem_addr = NULL;
20   int    i;
21
22   //Creation et initialisation
23   // du semaphore par le parent
24   sem_addr = sem_open("/mon_semaphore",
25           O_RDWR | O_CREAT, S_IRWXU, 0);
26   for(i = 0 ; i < N ; i++){
27     pid_fils[i] = fork();
28     if( 0 == pid_fils[i])
29       break;
30   }
31
32   if (i < N){ // Processus enfant
33     sem_addr = sem_open("/mon_semaphore", O_RDWR);
34     sem_wait(sem_addr);
35     fprintf(stdout,"Enfant %i OK\n",getpid());
36   } else if (i == N){ // Processus parent
37     pid_t pid;
38     sleep(5);
39
40     // Liberation des enfants
41     for(int i = 0; i < N; i++)
42       sem_post(sem_addr);
43
44     for(int i = 0; i < N; i++){
45       pid = wait(NULL);
```

```
46        fprintf(stdout,"Enfant %i fini \n",pid);
47      }
48      sem_unlink("/mon_semaphore");
49    }
50
51    exit(EXIT_SUCCESS);
52  }
```

En revanche, utiliser un sémaphore en variable globale aboutit à une solution **non fonctionnelle**. Voici un code implémentant cette « solution » :

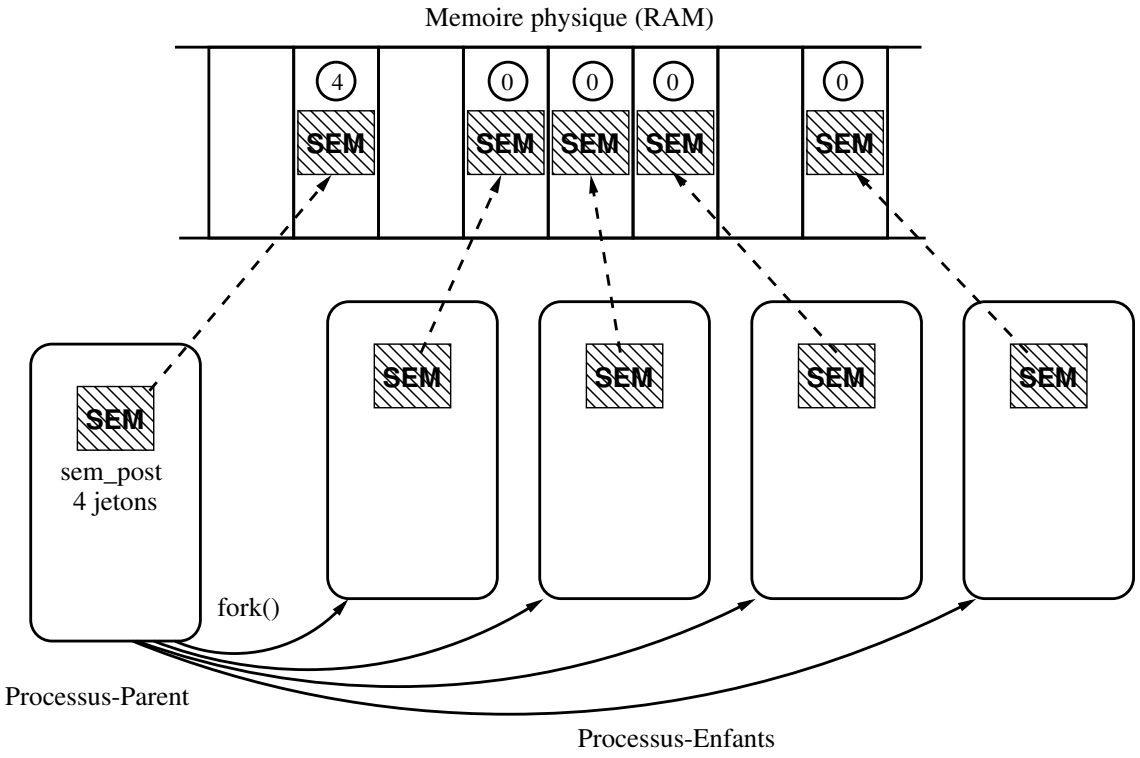
```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5  #include <sys/mman.h>
6  #include <string.h>
7  #include <signal.h>
8  #include <sys/wait.h>
9  #include <assert.h>
10 #include <stdalign.h>
11
12 #define N (10)
13
14 sem_t semaphore;
15
16 int main(int argc, char *argv[])
17 {
18   pid_t pid_fils[N];
19   int i;
20
21   sem_init(&semaphore, 1 , 0);
22
23   for(i = 0 ; i < N ; i++){
24     pid_fils[i] = fork();
25     if( 0 == pid_fils[i])
26       break;
27   }
28
29   if (i < N){
30     fprintf(stdout,"[%i] attente sur semaphore
         @%p\n",getpid(),&semaphore);
31     sem_wait(&semaphore);
32     fprintf(stdout,"Enfant %i OK\n",getpid());
33   } else if (i == N){
34     pid_t pid;
35     sleep(5);
36
37     for(int i = 0; i < N; i++)
38       sem_post(&semaphore);
39
```

```
40      for(int i = 0; i < N; i++){
41        pid = wait(NULL);
42        fprintf(stdout,"Enfant %i fini \n",pid);
43      }
44      sem_destroy(&semaphore);
45    }
46    exit(EXIT_SUCCESS);
47  }
```

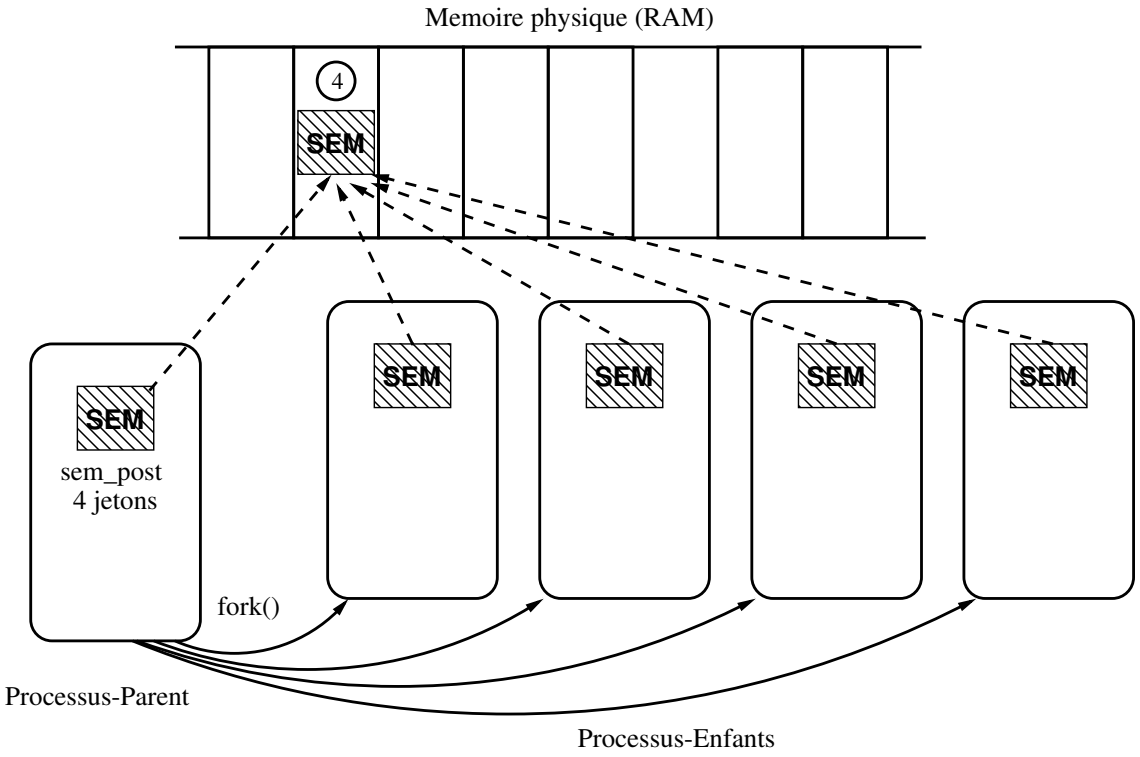
Pourtant, quand on exécute ce programme, tous les processus semblent accéder au même sémaphore comme le montre l'affichage de son adresse :

```
[1354329] attente sur semaphore @0x5e8b0e573040
[1354330] attente sur semaphore @0x5e8b0e573040
[1354331] attente sur semaphore @0x5e8b0e573040
[1354332] attente sur semaphore @0x5e8b0e573040
[1354333] attente sur semaphore @0x5e8b0e573040
[1354334] attente sur semaphore @0x5e8b0e573040
[1354335] attente sur semaphore @0x5e8b0e573040
[1354336] attente sur semaphore @0x5e8b0e573040
[1354337] attente sur semaphore @0x5e8b0e573040
[1354338] attente sur semaphore @0x5e8b0e573040
```

Examinons ce qui se passe dans la mémoire des différents processus. L'explication tient à l'emplacement dans la mémoire où est stocké le sémaphore. Dans le cas où le sémaphore est une variable globale du processus, on a la situation suivante :



Au moment du `fork()`, une *copie* du sémaphore du processus-parent (initialisé avec 0 jetons) est créée dans la mémoire (l'espace d'adressage) de chaque processus-enfant. Comme tous les processus possèdent un espace d'adressage avec la même structure, l'adresse de ce sémaphore est la même au sein de chaque processus (d'où l'affichage ci-dessus). Quand le processus-parent remet des jetons (opération `sem_post`) dans son sémaphore, ceux des processus-enfants ne sont pas modifiés et ces derniers restent bloqués dans l'appel à `sem_wait`.
Dans le cas où le sémaphore est placé dans une zone de mémoire partagée, c'est la situation suivante qui se produit :



Au moment du `fork()`, on duplique l'espace d'adressage du processus-parent et notamment la page-mémoire créée avec `mmap` et qui contient le sémaphore. Comme le mapping est déclaré comme partagé (option `MAP_SHARED`), alors cette page-mémoire est partagée avec les processus-enfants. Donc, quand le processus parent remet des jetons dans le sémaphore, ces jetons sont disponibles pour les processus-enfants et ils sont débloqués.

Exercice 63 (⋆⋆)

1. Écrivez un programme qui crée deux threads qui se partagent un buffer d'**un** caractère :
   — le premier de ces threads sera le **producteur**, qui écrit dans le caractère une séquence de lettres de l'alphabet. Ce thread producteur ne peut écrire dans le buffer que s'il y a de la place de libre.
   — l'autre de ces threads sera le **consommateur**. Il lit un caractère à la fois depuis le buffer partagé. Le consommateur ne peut lire dans le buffer que s'il y a effectivement quelque chose à lire, c'est-à-dire quand le producteur y a écrit un caractère. Une fois la lecture effectuée, la place est considérée comme étant libérée (pour être utilisée par le producteur).

2. Généralisez ce programme avec un nombre pair de threads et un buffer partagée contenant NSLOTS places. La moitié de ces threads sont des **producteurs**, qui écrivent chacun un caractère à la fois dans le buffer partagé. l'autre moitié de ces threads sont des **consommateurs** qui lisent chacun un caractère à la fois dans le buffer.
   Vous ferez attention à bien synchroniser les exécutions des différents producteurs et consommateurs. Également, vous veillerez à ce que les données dans le buffer restent cohérentes et mettrez en place une gestion adéquate des places libres dans le buffer (ex : gestion circulaire, gestion du premier emplacement libre/à lire, etc.)

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/wait.h>
7  #include <errno.h>
8  #include <assert.h>
9  #include <pthread.h>
10 #include <semaphore.h>
11
12 #define NTHREADS (8)
13 #define NSLOTS   (5) // Number of slots available in buffer
14 #define NREPS    (7)
15
16 sem_t sem_prod;
17 sem_t sem_cons;
18 sem_t sem_array;
19
20 int first_write = 0;
21 int first_read  = 0;
22
23 int shared_buffer[NSLOTS] = {0};
24
25 void *producteur (void *arg){
26   int  idx = *((int*)arg);
27   char c   = ('a' + idx);
28
29   for(int i = 0 ; i < NREPS ; i++){
30     sem_wait(&sem_prod);  // Wait for an empty slot
31
32     sem_wait(&sem_array); // Access array
33     shared_buffer[first_write] = c;  //write in buffer
34     fprintf(stdout,"Producer[%i] wrote %c @ index
         %i\n",idx,c,first_write);
35     first_write++;   // Update position
36     first_write %= NSLOTS;
37     sem_post(&sem_array);
38
```

```
39      sem_post(&sem_cons); // release consumer
40    }
41
42    pthread_exit(NULL);
43  }
44  void *consommateur (void *arg){
45    int idx = *((int*)arg);
46
47    for(int i = 0 ; i < NREPS ; i++){
48      sem_wait(&sem_cons); //Wait for char in slot
49
50      sem_wait(&sem_array); // Access array
51      char c = shared_buffer[first_read]; //read in buffer
52      fprintf(stdout,"Consumer[%i]read %c @ index %i / nrep
          %i\n",idx,c,first_read,i);
53      shared_buffer[first_read] = 0; // reset array
54      first_read++;  // Update position
55      first_read %= NSLOTS;
56      sem_post(&sem_array);
57
58      sem_post(&sem_prod); // notify producer that a free slot
          is available
59    }
60
61    pthread_exit(NULL);
62  }
63
64  int main(int argc, char *argv[])
65  {
66    void * (*f_ptr_array[])(void *) = {producteur,
        consommateur};
67    pthread_t tids[NTHREADS];
68    int       args[NTHREADS];
69
70    sem_init(&sem_prod,  0, NSLOTS);
71    sem_init(&sem_cons,  0, 0);
72    sem_init(&sem_array, 0, 1);
73
74    for(int i = 0 ; i < NTHREADS ; i++){
75      args[i] = i;
76      pthread_create(tids+i, NULL, f_ptr_array[i%2], (void
          *)(args+i));
77    }
78
79    for(int i = 0 ; i < NTHREADS ; i++){
80      pthread_join(tids[i],NULL);
81    }
82
83    sem_destroy(&sem_prod);
```

```
84    sem_destroy(&sem_cons);
85    sem_destroy(&sem_array);
86
87    exit(EXIT_SUCCESS);
88  }
```

8.2 Les verrous

Exercice 64 (⋆)
Un sémaphore initialisé avec un unique jeton est-il équivalent à un verrou ?

Correction

Non, car le sémaphore ne garantit pas l'exclusion mutuelle systématiquement, même avec un unique jeton. Par exemple, un flot prend le jeton du sémaphore et continue son exécution pour accéder à de la section critique. À ce moment, un deuxième flot pourrait remettre un jeton dans le sémaphore et permettre à un troisième flot de rentrer à son tour dans cette section critique.

Exercice 65 (⋆⋆⋆)
Quelles conditions supplémentaires sont-elles nécessaires pour garantir l'exclusion mutuelle avec un sémaphore ?

Correction

Les deux conditions suivantes sont nécessaires :

— le sémaphore doit contenir un unique jeton au maximum, en toutes circonstances ;
— seul le flot qui prend le jeton est autorisé à le remettre dans le sémaphore (le jeton n'est pas « magique »).

Exercice 66 (⋆)

— Une variable partagée entre des threads et lue/écrite par chacun des threads peut-elle être considérée comme de la section critique de code ?
— Un mécanisme de protection est-il indispensable ?

Correction

— Il s'agit de section critique dans tous les cas, sauf si la variable n'est accédée qu'en écriture et qu'aucun thread ne la modifie.
— Ça dépend du type de la variable : pour une variable atomique, pas besoin de la protéger (avec un mutex, par exemple).

Exercice 67 (⋆)
Une variable partagée entre des threads où un seul thread lit la variable tandis que les autres la modifient peut-elle être considérée comme de la section critique de code (au niveau **des** écritures et de **la** lecture) ?

Correction

Oui dans tous les cas (lecture et écriture) : par exemple, `if (var > 0) code` est une section critique, parce que sans mécanisme de synchronisation, il est possible que la variable `var` soit modifée entre le test (`if`) et l'exécution du bloc de code. Il y aura un accès à une section de code alors que la variable n'a pas la bonne valeur.

Exercice 68 ($\star$)

Écrivez un programme où le processus crée n threads qui se partagent un tableau de m entiers (ce tableau pourra être une variable globale du programme pour simplifier les choses) et qui calculent le minimum du tableau (ce minimum pourra également être stocké dans une variable globale du programme).

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <pthread.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <stdint.h>
8  #include <inttypes.h>
9  #include <semaphore.h>
10 #include <sys/time.h>
11 #include <string.h>
12 #include <assert.h>
13
14 #define M (1000000)
15
16 typedef struct th_arg{
17   int64_t         *base_ptr;
18   int64_t          num_elems;
19   int64_t         *global_min;
20   pthread_mutex_t *lock_ptr;
21 } th_arg_t;
22
23 void *sort(void *arg)
24 {
25   th_arg_t *myarg  = ((th_arg_t *)arg);
26   int64_t  *ptr    = myarg->base_ptr;
27   int64_t   nelems = myarg->num_elems;
28   int64_t   min    = *ptr; // first element of subarray
29
30   double    time   = 0.0;
31   struct timeval t1,t2;
32
33   gettimeofday(&t1,NULL);
34   for(int64_t i = 1 ; i < nelems ; i++)
35     if( ptr[i] < min)
36       min = ptr[i];
37   gettimeofday(&t2,NULL);
38
39   time = (t2.tv_sec - t1.tv_sec) + (t2.tv_usec -
       t1.tv_usec)/1000000.0;
40   fprintf(stdout,"Time spent in subsort: %f sec \n",time);
```

```
41
42    pthread_mutex_lock(myarg->lock_ptr);
43    if( min <  *(myarg->global_min))
44      *(myarg->global_min) = min ;
45    pthread_mutex_unlock(myarg->lock_ptr);
46
47    pthread_exit((void *)min);
48  }
49
50  int main(int argc, char *argv[])
51  {
52    int64_t   *garray = NULL;
53    int64_t   *marray = NULL;
54    pthread_t *tid = NULL;
55    th_arg_t  *args = NULL;
56    pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
57    int num_threads = 10;
58    int64_t  min;
59    int64_t  global_min;
60    struct timeval t1,t2;
61    float time = 0.0;
62    int seed   = 666;
63    memset(&t1,0,sizeof(struct timeval));
64    memset(&t2,0,sizeof(struct timeval));
65
66    if(3 == argc) {
67      num_threads = atoi(argv[1]);
68      seed        = atoi(argv[2]);
69    }
70
71    fprintf(stdout,"creating %i threads\n",num_threads);
72    srand(seed);
73
74    tid    = (pthread_t *)malloc(num_threads *
          sizeof(pthread_t));
75    args   = (th_arg_t *)malloc(num_threads * sizeof(th_arg_t));
76    marray = (int64_t *)malloc(num_threads * sizeof(int64_t));
77    garray = (int64_t *)malloc(M * sizeof(int64_t));
78
79    for(int64_t i = 0 ; i < M ; i++)
80      garray[i] = rand();
81
82    global_min = garray[0];
83
84    gettimeofday(&t1,NULL);
85    for(int i = 0 ; i < num_threads ; i++){
86      args[i].base_ptr   = garray + i*(M/num_threads);
87      args[i].num_elems  = M/num_threads;
88      args[i].global_min = &global_min;
```

```
89      args[i].lock_ptr    = &mutex;
90    }
91    args[num_threads-1].num_elems += M%num_threads;
92
93    for(int i = 0 ; i < num_threads ; i++)
94      pthread_create(&tid[i],NULL,sort,(void *)&(args[i]));
95
96    for(int i = 0 ; i < num_threads ; i++)
97      pthread_join(tid[i],(void *)&marray[i]);
98
99    gettimeofday(&t2,NULL);
100
101   /* sanity check */
102   min = marray[0];
103   for(int i = 1 ; i < num_threads ; i++)
104     if(marray[i] < min)
105       min = marray[i];
106   assert(min == global_min);
107
108   time  = (t2.tv_sec - t1.tv_sec) + (t2.tv_usec -
        t1.tv_usec)/1000000.0;
109   fprintf(stdout,"Total time spent: %f sec\n",time);
110
111   pthread_mutex_destroy(&mutex);
112
113   free(tid);
114   free(args);
115   free(marray);
116   free(garray);
117
118   fprintf(stdout,"End of main: Min  is %" PRId64
        "\n",global_min);
119
120   return 0;
121 }
```

8.3 Les variables de condition

Exercice 69 $(\star)$

Comment peut-on qualifier la condition logique utilisée ?

Correction

Il s'agit d'une section critique de code qui doit être protégée par un verrou, aussi bien en lecture qu'en écriture.

8.3.1 Implémentation d'un moniteur par l'exemple

Exercice 70 $(\star\star\star)$

Cette implémentation de moniteur est malheureusement fausse.

— Où se trouve l'erreur ?

— Comment faut-il la corriger ?

Correction

Le problème est le suivant : quand un flot est réveillé — par un autre flot qui lui signale la réalisation de la condition logique — et devient le propriétaire du verrou, il a la garantie qu'il est le seul à accéder à la condition logique (exclusion mutuelle) **mais il n'a pas la garantie que la condition logique est toujours vérifiée.** Pourquoi ? Parce qu'il est possible qu'entre son réveil et sa (tentative de) prise de verrou, un autre flot ait réussi à :

1. prendre le verrou avant
2. modifier la condition logique, la rendant potentiellement invalide
3. libérer le verrou

Or, d'un point de vue sémantique, le flot n'est pas censé accéder à la section critique si la condition logique n'est pas vérifiée. Il est donc indispensable de revérifier l'état de la condition logique avant d'accéder effectivement à la section critique. **Il faut donc remplacer le test `if` par un `while`.** Deux cas peuvent se produire :

Cas 1 : la condition est vérifiée, donc le flot ne rappelle pas la fonction `pthread_cond_wait`. Il rentre légitimement dans la section critique et libère le verrou quand il en sort. Ce flot a le droit de libérer ce verrou car il en est devenu le propriétaire juste avant de sortir de son appel précédent à `pthread_cond_wait`.

Cas 2 : la condition n'est pas vérifiée donc le flot rappelle la fonction `pthread_cond_wait`. Il est mis en sommeil et libére le verrou dont il est devenu propriétaire juste avant de sortir de son appel précédent à `pthread_cond_wait`.

Il existe plusieurs types de moniteurs.Les moniteurs implémentables avec les objets de la bibliothèque pthread sont appelé moniteur MESA. Les moniteurs de Hoare utilisent un `if` plutôt qu'un `while` pour tester la condition logique parce que dans ce cas, c'est le thread qui est réveillé qui devient le nouveau propriétaire du verrou. Par conséquent, la condition logique n'est pas modifiée entre le réveil du flot et sa prise (réussie) de verrou.

Exercice 71 (⋆⋆⋆)
Implémentez des sémaphores avec des moniteurs : commencez par définir la structure de donnée adéquate puis implémentez les opérations `INIT`, `P()` et `V()`.

Correction

Solution basique.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <sys/types.h>
5  #include <sys/stat.h>
6  #include <stdint.h>
7  #include <inttypes.h>
8  #include <sys/time.h>
9  #include <string.h>
10 #include <assert.h>
11 #include <sys/wait.h>
12 #include <string.h>
13 #include <fcntl.h>
14 #include <unistd.h>
15 #include <sys/syscall.h>
```

```
16
17 typedef struct my_sem_s
18 {
19   int tokens;
20   pthread_mutex_t mutex;
21   pthread_cond_t cond;
22 } my_sem_t;
23
24 int my_sem_init(my_sem_t *sem, int tokens)
25 {
26   sem->tokens = tokens;
27   pthread_mutex_init(&sem->mutex, NULL);
28   pthread_cond_init(&sem->cond, NULL);
29   return 0;
30 }
31
32 int my_sem_P(my_sem_t *sem)
33 {
34   pthread_mutex_lock(&sem->mutex);
35   while( sem->tokens == 0){
36     pthread_cond_wait(&sem->cond,&sem->mutex);
37   }
38   sem->tokens--;
39   pthread_mutex_unlock(&sem->mutex);
40   return 0;
41 }
42
43 int my_sem_V(my_sem_t *sem)
44 {
45   pthread_mutex_lock(&sem->mutex);
46   sem->tokens++;
47   pthread_cond_broadcast(&sem->cond);
48   pthread_mutex_unlock(&sem->mutex);
49   return 0;
50 }
51
52 void *f(void *arg)
53 {
54   my_sem_t *sem = (my_sem_t *)arg;
55   pid_t base_rank = getpid();
56   pid_t my_rank  = syscall(SYS_gettid);
57
58   fprintf(stdout,"[%i][%i] Taking semaphore
       ...\n",base_rank,my_rank);
59   my_sem_P(sem);
60   fprintf(stdout,"[%i][%i] Done \n",base_rank,my_rank);
61
62   pthread_exit(NULL);
63 }
```

```
64
65
66  int main(int argc, char *argv[])
67  {
68    int num_threads = 10;
69    pthread_t *tids = malloc(num_threads*sizeof(pthread_t));
70    my_sem_t sem;
71
72    my_sem_init(&sem, 0);
73
74    for(int i = 0 ; i < num_threads ; i++){
75      pthread_create(&tids[i],NULL,f,(void *)&sem);
76    }
77
78    for(int i = 0 ; i <  num_threads  ; i++){
79      sleep(1);
80      my_sem_V(&sem);
81      fprintf(stdout,"[MASTER] token in %i \n",i);
82    }
83
84    for(int i = 0 ; i < num_threads ; i++){
85      pthread_join(tids[i],NULL);
86    }
87
88    free(tids);
89
90    exit(EXIT_SUCCESS);
91  }
```

Solution avec support de shared et wakeups. Inspiré de Make your own semaphores, Little Book of Semaphores, p. 291.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <sys/mman.h>
5  #include <string.h>
6  #include <sys/types.h>
7  #include <unistd.h>
8  #include <sys/types.h>
9  #include <sys/wait.h>
10
11 #define NB_FILS  (10)
12 #define MAP_SIZE (4096)
13
14 typedef struct my_sem_s
15 {
16   int tokens;
17   int wakeups;
18   pthread_mutex_t mutex;
19   pthread_cond_t cond;
```

```
20 } my_sem_t;
21
22 int my_sem_init(my_sem_t *sem, int shared, int tokens)
23 {
24   sem->tokens = tokens;
25   sem->wakeups = 0;
26
27   if (shared == 0){
28     pthread_mutex_init(&sem->mutex, NULL);
29     pthread_cond_init(&sem->cond, NULL);
30   } else {
31     pthread_condattr_t cond_attr;
32     pthread_condattr_init(&cond_attr);
33     pthread_condattr_setpshared(&cond_attr,
          PTHREAD_PROCESS_SHARED);
34     pthread_cond_init(&sem->cond, &cond_attr);
35
36     pthread_mutexattr_t attr;
37     pthread_mutexattr_init(&attr);
38     pthread_mutexattr_setpshared(&attr,
          PTHREAD_PROCESS_SHARED);
39     pthread_mutex_init(&sem->mutex, &attr);
40   }
41   return 0;
42 }
43
44 int my_sem_P(my_sem_t *sem)
45 {
46   pthread_mutex_lock(&sem->mutex);
47   sem->tokens--;
48   if ( sem->tokens < 0) {
49     do {
50       pthread_cond_wait(&sem->cond,&sem->mutex);
51     } while ( sem->wakeups < 1);
52     sem->wakeups--;
53   }
54   pthread_mutex_unlock(&sem->mutex);
55   return 0;
56 }
57
58
59 int my_sem_V(my_sem_t *sem)
60 {
61   pthread_mutex_lock(&sem->mutex);
62   sem->tokens++;
63   if ( sem->tokens <= 0) {
64     sem->wakeups ++;
65     pthread_cond_signal(&sem->cond);
66   }
```

```
67    pthread_mutex_unlock(&sem->mutex);
68    return 0;
69  }
70
71  int main(int argc, char *argv[])
72  {
73      my_sem_t *semaphore = mmap(NULL, MAP_SIZE, PROT_READ |
            PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
74      pid_t fils[NB_FILS];
75
76      memset(semaphore, 0, sizeof(my_sem_t));
77
78      my_sem_init(semaphore, 1, 0);
79
80      for (int i = 0; i < NB_FILS; i++){
81        fils[i] = fork();
82        if (fils[i] == 0){
83    my_sem_P(semaphore);
84    fprintf(stdout,"Child number %d\n", i);
85    exit(EXIT_SUCCESS);
86        }
87      }
88
89      fprintf(stdout,"Process creation done, sleeping ...\n");
90      sleep(5);
91      fprintf(stdout,"Done\n");
92
93      for (int i = 0; i < NB_FILS; i++){
94        my_sem_V(semaphore);
95      }
96
97      munmap(semaphore, MAP_SIZE);
98
99      for (int i = 0; i < NB_FILS; i++){
100       wait(NULL);
101     }
102
103     exit(EXIT_SUCCESS);
104 }
```

8.4 Les barrières de synchronisation

Exercice 72 (⋆⋆)

Implémentez une *barrière de synchronisation entre threads* à l'aide d'un moniteur. Pour se faire, il vous faudra :

— définir une structure de données `my_barrier_t`

— écrire la fonction d'initialisation :

```
int my_barrier_init(my_barrier_t *my_barrier, const int num_threads);
```

— écrire la fonction d'attente dans la barrière :

```
int my_barrier_wait(my_barrier_t *my_barrier);
```

Correction

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <pthread.h>
5
6  typedef struct {
7    pthread_mutex_t lock;
8    pthread_cond_t  cond;
9    int             value;
10 } my_barrier_t;
11
12 int my_barrier_init(my_barrier_t *barrier, unsigned count)
13 {
14   pthread_mutex_init(&(barrier->lock),NULL);
15   pthread_cond_init(&(barrier->cond),NULL);
16   barrier->value = count;
17   return 1;
18 }
19
20 int my_barrier_wait(my_barrier_t *barrier)
21 {
22   pthread_mutex_lock(&(barrier->lock));
23   if(--(barrier->value) <= 0)
24      pthread_cond_broadcast(&(barrier->cond));
25   else while ((barrier->value) > 0)
26     pthread_cond_wait(&(barrier->cond),&(barrier->lock));
27   pthread_mutex_unlock(&(barrier->lock));
28   return 1;
29 }
30
31 int my_barrier_destroy(my_barrier_t *barrier)
32 {
33   pthread_mutex_destroy(&(barrier->lock));
34   pthread_cond_destroy(&(barrier->cond));
35   return 1;
36 }
37
38 void *function1(void *arg)
39 {
40   my_barrier_t *barrier = (my_barrier_t *)arg;
41
42   printf("Je rentre dans la barriere ...\n");
43   my_barrier_wait(barrier);
44   printf("Je continue ...\n");
```

```
45
46    pthread_exit(NULL);
47  }
48
49  int main(int argc, char **argv)
50  {
51    int           num_thread = argc > 1 ? atoi(argv[1]) : 4;
52    my_barrier_t  barrier;
53    pthread_t     tid[num_thread];
54
55    my_barrier_init(&barrier,num_thread);
56
57    for(int idx = 0 ; idx < num_thread ; idx++){
58      pthread_create(&tid[idx],NULL,function1,(void *)&barrier);
59      sleep(2);
60    }
61
62    for(int idx = 0 ; idx < num_thread ; idx++)
63      pthread_join(tid[idx],NULL);
64
65    my_barrier_destroy(&barrier);
66    exit(EXIT_SUCCESS);
67  }
```

Exercice 73 (⋆⋆⋆)

Dans l'exercice précédent, vous avez implémenté une barrière de synchronisation inter-threads avec un moniteur.

— Trouvez le problème posé par la solution basée sur un moniteur.
— Proposez un correctif. Est-il fonctionnel dans tous les cas ?

Correction

Une barrière est censée être utilisable plusieurs fois d'affilée. Or, si la solution proposée n'est valable pour une utilisation *one-shot*. En effet, le champ `value` de la structure proposée est décrémenté à chaque fois qu'un thread entre dans la barrière et les threads en attente sont libérés quand cette valeur tombe à 0. Cette valeur n'est jamais réinitialisée quand le dernier threads rentre dans la barrière et donc une nouvelle utilisation se solderait par un blocage de threads qui voudraient à nouveau rentrer dans la barrière.

Cependant, réinitialiser la valeur (avec le nombre initial, passé en argument à la fonction `init`) n'est pas suffisant et des interblocages pourraient survenir. Prenons un exemple concret : la barrière est initialisée pour n threads et est utilisée une première fois : $n-1$ threads rentrent dans la barrière et sont mis en sommeil (appel à `pthread_cond_wait`). Quand le dernier thread rentre dans la barrière, il signale que la condition logique est vérifiée (i.e. `value == 0`) et réinitialise cette valeur (`value = n`). Mais les autres threads qui sont réveillés par ce dernier thread vont retester la condition logique, qui n'est plus vérifiée (puisque la valeur a été réinitialisée) et donc vont être remis en sommeil aussitôt.

Une solution serait de comptabiliser le *numéro de séquence d'utilisation de la barrière* afin de déterminer combien un thread a utilisé la barrière et dans quelle utilisation il se trouve. Mais cela complexifie la conception...

En réalité, il est difficile (même si possible) d'implémenter *correctement* une barrière de

synchronisation avec les outils déjà étudiés (verrous, moniteurs, sémaphores) donc il est préférable d'utiliser les objets disponibles dans la bibliothèque pthread.

Exercice 74 (⋆)
Écrivez un programme dans lequel le thread principal crée 10 threads (qui affichent une chaîne de caractère par exemple) mais tous ces nouveaux threads doivent attendre que le dernier soit créé pour effectuer leur affichage.

Correction

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/wait.h>
#include <errno.h>
#include <assert.h>
#include <pthread.h>

#define NTHREADS (10)

pthread_barrier_t barrier;

void *thread_func(void *arg){
  sleep(5);
  pthread_barrier_wait(&barrier);

  fprintf(stdout,"Hello from %i\n",*(int *)arg);

  pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
  pthread_t tids[NTHREADS];
  int       args[NTHREADS] = {0};

  pthread_barrier_init(&barrier, NULL, NTHREADS);

  for(int i = 0 ; i < NTHREADS ; i++){
    args[i] = i;
    pthread_create(tids+i, NULL, thread_func, args+i);
  }

  for(int i = 0 ; i < NTHREADS ; i++){
    pthread_join(tids[i],NULL);
  }

  pthread_barrier_destroy(&barrier);
```

```
41
42    exit(EXIT_SUCCESS);
43 }
```

8.5 Pour aller (vraiment) plus loin : synchronisation et variables atomiques